Computational studies on aromaticity, Lewis acid-base adducts and applications using Machine Learning methods

Report on work carried during
the

Summer Internship 2025

in

Presented by

Jeeva K

2nd Year, 23MS234
Indian Institute of Science Education and Research (IISER), Kolkata

Under supervision of:

Dr. Debasis Koley

Department of Chemical Sciences
Indian Institute of Science Education and Research (IISER), Kolkata

Declaration by the Student

I, Jeeva K, hereby certify that I have performed all calculations for this report titled **"Computational studies on aromaticity, Lewis acid-base adducts and applications using Machine Learning methods"**, based on my personal study and effort. This report is based on three research papers, which are duly acknowledged at the end. In addition to these references, I have conducted further independent work. All the calculations and analyses presented in this report were carried out independently and yielded consistent results.

Date: 21/07/2025

.....

Place: IISER Kolkata

Jeeva K

Department of Chemical Sciences

Indian Institute of Science Education and Research

Kolkata, Mohanpur - 741246

West Bengal, India

.....

Acknowledgments

I would like to express my sincere gratitude to my mentor, Prof. Debasis Koley, for his invaluable assistance throughout this project. His guidance at every step helped me learn many new things, making his presence during the internship incredibly valuable.

I would also like to thank Miss. Leya Elsa George and other CCMM members for their guidance throughout the project. I am also grateful to IISER Kolkata and CCMM lab for providing the necessary facilities for this work. Additionally, I extend my heartfelt thanks to my friends and parents for their unwavering support and encouragement.

Date: 21/07/2025

.....

Place: IISER Kolkata

Jeeva K

Department of Chemical Sciences

Indian Institute of Science Education and Research

Kolkata, Mohanpur - 741246

West Bengal, India

.....

Contents

Abstract

This report involves three computational studies that integrate quantum chemical calculations and machine learning to solve important challenges in molecular chemistry and chemical data analysis. In the first experiment, the concept of aromaticity is analyzed by calculating resonance energies using thermodynamic electronic energies of different homocyclic and heterocyclic hydrocarbons and ions using Gaussian 16. Geometry optimizations and frequency analyses were performed using semi-empirical AM1 methods, followed by Hartree-Fock single point energy calculations with the 3-21G(*) basis set to assess aromatic stabilization.

The second experiment investigates the binding interactions of various Lewis acid-base adducts involving Boron & aluminum hydrides and halides as acid and nitrogen & phosphorus containing bases. Computational methods identical to those in the first study were employed, with modifications in basis sets to evaluate electronic and steric effects. Key parameters such as HOMO-LUMO energies, vertical electron attachment energy, reorganization energy, and proton affinity were analyzed to determine relative acid-base strengths and reactivity.

The third experiment applies machine learning techniques to analyze a physicochemical dataset of red and white wines quality dataset. Using Interactive Python and Jupyter notebooks, the study covers data pre-processing, visualization, and supervised learning models to derive patterns and build predictive frameworks. All three experiments show the utility of computational chemistry and data science in understanding molecular properties and handling experimental datasets.

1 Project: I - Quantitative Thermodynamic Descriptions of Aromaticity

[2]

1.1 Introduction

This study focuses on understanding the concept of aromaticity of organic compounds and verifying the aromaticity rules various homocyclic and heterocyclic hydrocarbons and hydrocarbon ions by finding their resonance energy, using computational calculations in *Gaussian-16* software integrated with the *GaussView-6* graphical interface.

The computational procedure involves constructing the organic molecule and Geometry Optimization+Frequency *(Opt+freq)* using *ab-initio* semi-empirical AM1 calculation and followed by single point *(sp)* energy calculations on the optimized structures using the Hartree-Fock method with the HF 3-21G(*) basis set.

1.2 Theory

1.2.1 Aromaticity

Aromatic Compounds are a crucial part of organic chemistry. The concept of Aromaticity is defined by Huckel's rule. For a molecule to be aromatic it should follow three rules.

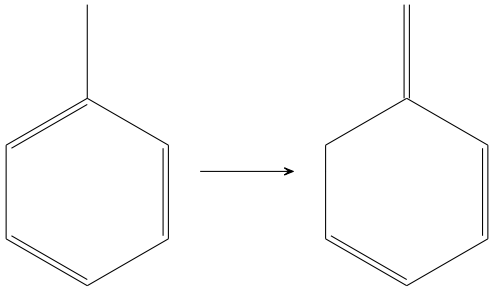
- **Cyclic:** The molecule must be forming a closed loop.
- **Planar:** The molecule must have sp^2 hybridized orbitals, so that p-orbitals can overlap continuously.
- **Conjugation:** The ring must have alternate double bonds for conjugation of π electrons;such that it is fully conjugated, allowing delocalization of π-electrons.
- **$(4n + 2)$ π-electrons:** The molecule must contain $(4n + 2)\pi-$ electrons in the conjugated system, where $n = (0, 1, 2, ...)$.

Property	**Aromatic Compounds**	**Anti-Aromatic Compounds**	**Non-Aromatic Compounds**
Planarity	Planar	Planar	May or may not be planar
Conjugation	Fully conjugated (alternating π bonds)	Fully conjugated	Not fully conjugated
Hückel's Rule	Follows $4n + 2$ π electrons (n = 0, 1, 2, ...)	Follows $4n$ π electrons (n = 1, 2, 3, ...)	Doesn't follow Hückel's rule
Stability	Highly stable due to delocalization of electrons	Highly unstable due to electron repulsion	No special stabilization
Delocalization of π Electrons	Extensive and continuous delocalization	Delocalized but leads to instability	No continuous delocalization
Resonance Energy	Higher positive value	Higher negative value	Nearly equals to zero

Table 1: Comparison between Aromatic, Anti-Aromatic, and Non-Aromatic Compounds

1.2.2 Resonance Energy

The resonance energy explains the stability of aromatic compounds. We have used *Isodesmic Isomerization reaction* for calculating the resonance energy of molecules. An Isodesmic Isomerization reaction is a hypothetical chemical reaction where the number and type of chemical bonds are the same on both sides of the equation. The isodesmic structures ($A\&B$) shown in Figure 1, of the desired molecules were constructed and their electronic energy was computed, and the results were used to classify the compounds as aromatic, non-aromatic, or anti-aromatic.



Resonance Hybrid (**A**) Isomer (**B**)

Figure 1: Example of Isodesmic Isomerization reaction.

1.2.3 Computational Calculation

Optimization + Frequency calculation: This finds the molecular geometry with the lowest possible energy, which is the global minimum on the potential energy surface (PES) with only positive frequency values.

Single Point Energy Calculation: This solves the Schrodinger equation using the approximation technique used by the given basis and finding the Total Electronic Energy of the input molecule at given geometry.

Basis set is a collection of mathematical functions which provide the tool to build the approximation to solve the Schrodinger equation by representing the molecular orbitals as a linear combination of basis functions.

In computational chemistry, selecting an appropriate basis set is essential for balancing accuracy, time and computational cost. Geometry optimization and frequency calculations were performed using the semi-empirical AM1 method due to its efficiency for small and simple molecules. A more accurate single point energy calculation was then carried out using the Hartree-Fock method with the significantly higher-level 3-21G(*) basis set. All the calculations are done within seconds and the results are comparable with the higher basis-set calculation data.

1.3 Observation and Results

The Resonance energy was calculated for the following molecules:
6 homocyclic molecules: Benzene, Naphthalene, cyclobutadiene, Cyclooctatetraene, Cycloheptatrienyl cation and Cycloheptatrienyl anion.
3 heterocyclic molecules: Furan, Pyridine, Thiophene.

The thermodynamic electronic energies of the isodesmic isomers of the given molecules were calculated. The resonance energy of each molecule was determined as the difference between the electronic energy of the isomer and that of the resonance hybrid, as shown below:

$$E_{\text{Resonance}} = E_{B\ (Isomer)} - E_{A\ (ResonanceHybrid)}$$

The results are summarized in the tables table 2,table 3, table 4.
We can compare the Stability and Aromaticity of the molecules using resonance energy. As a whole, we can write the descending order of stability of organic compounds as:

Benzene $\gtrsim$ Pyridine > Cycloheptatrienyl cation > Napthalene > Thiophene > Furan > Cyclooctatetraene > Cycloheptatrienyl anion > Cyclobutadiene

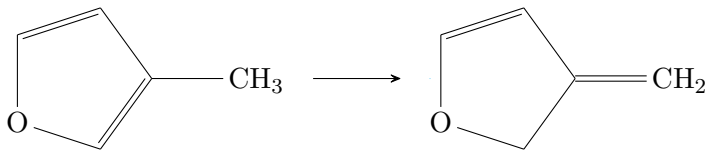
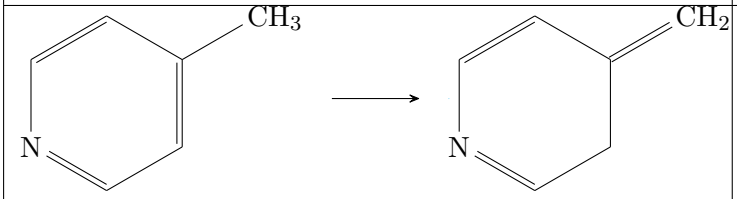
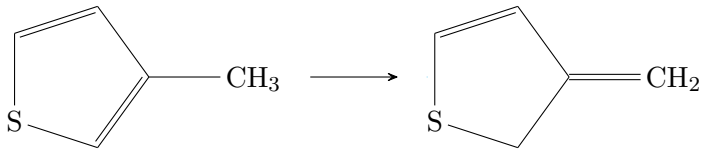
Resonance structures (A → B)	**Energy of A (a.u.)**	**Energy of B (a.u.)**	ΔE_{sp} **(kcal/-mol)**
	-266.16	-266.15	11.23
	-284.13	-284.07	36.33
	-587.41	-587.38	15.56

Table 2: Resonance Energy calculation for heterocyclic compounds using their Isodesmic Isomerisation reaction

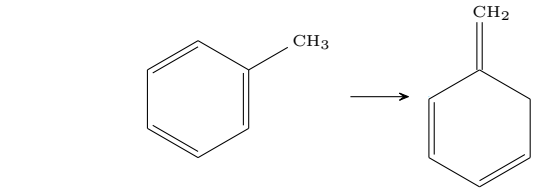
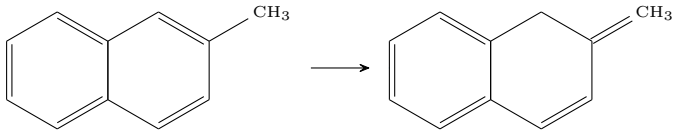
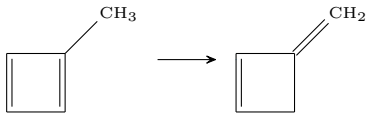
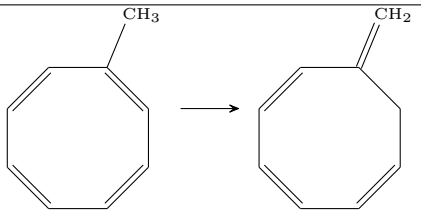
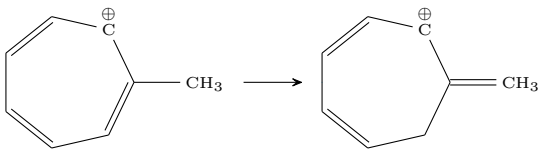
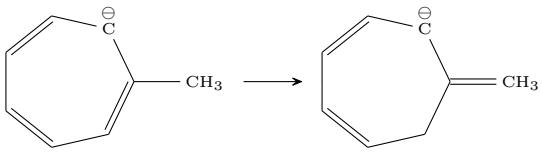
Resonance structures (A → B)	**Energy of A (a.u.)**	**Energy of B (a.u.)**	ΔE_{sp} **(kcal/-mol)**
	-268.23	-268.17	37.66
	-420.03	-419.99	25.31
	-191.59	-191.66	-41.18
	-344.62	-344.62	0.55
	-306.21	-306.16	32.47
	-306.33	-306.36	-19.28

Table 3: Resonance Energy calculation for homocyclic hydrocarbons and hydrocarbon ions using their Isodesmic Isomerisation reaction

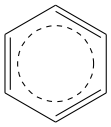
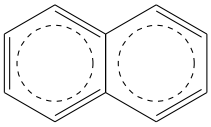
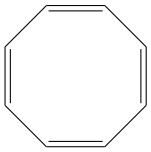
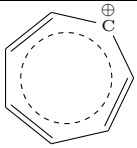
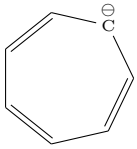
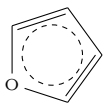
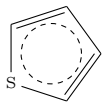
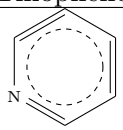
Structure	**Number of π electrons**	**Geometry**	**Resonance Energy (kcal/-mol)**	**Aromaticity**
 Benzene	6	Planar	37.66	Aromatic
 Naphthalene	10	Planar	25.31	Aromatic
 Cyclobutadiene	4	Planar	-41.18	Anti-aromatic
 Cyclooctatetraene	8	Non-planar	0.55	Non-aromatic
 Cycloheptatrienyl cation	6	Planar	32.47	Aromatic
 Cycloheptatrienyl anion	6	Planar	-19.28	Anti-aromatic
 Furan	6	Planar	11.23	Aromatic
 Thiophene	6	Planar	15.56	Aromatic
 Pyridine	6	Planar	36.33	Aromatic

Table 4: Aromaticity in conjugated homocyclic and heterocyclic compounds

1.4 Conclusion and Discussion

This study demonstrates the utility of computational chemistry in analyzing molecular structures and predicting chemical behavior. By applying electronic structure methods, researchers can gain insights into properties and bonding patterns that are often difficult to observe experimentally. The combination of molecular modeling and quantum mechanical calculations provides a framework for understanding chemical systems at the electronic level.

The use of isodesmic reactions in this experiment allowed for a practical and efficient approach to quantifying aromaticity. Since the method involves comparing structurally similar species with identical bonding patterns, it minimizes systematic errors and offers reliable energy differences. The calculations, although based on relatively small basis sets, produced meaningful results in a short computation time—showing strong agreement with theoretical expectations. This approach requires only the optimized geometry and electronic energy of two structures per molecule, making it suitable for a wide range of compounds. While this experiment focused on aromaticity, the same computational strategies are broadly applicable to various molecular systems as we are going to see in section 2.3 .

In summary, this experiment highlights how computational tools like Gaussian and GaussView can model molecular structures, interpret bonding characteristics, and support chemical predictions. It reinforces the value of integrating theoretical methods with experimental insight to advance understanding in modern chemical research.

.....

2 Project : II - A Computational Analysis of Lewis Acids and Bases: Optimizing Electronic Properties and Reactivity

[1]

2.1 Introduction

The study investigates the relative strengths of various Lewis acids and bases by calculating the binding energies of their adducts and analyzing key factors influencing these interactions. Specifically, adducts formed between boron and aluminum based hydrides and halides as Lewis acids, nitrogen and phosphorus containing bases were examined using *ab-initio* quantum mechanical methods. The same methodology and calculations from project 1 were performed with a different basis set Hartree-Fock HF 3-21G in Gaussian 16, with visualization in GaussView 6.

The relative acid and base strengths were evaluated through chemical hardness, determined from HOMO and LUMO energies. Additional parameters analyzed include vertical electron attachment energy, reorganization energy, proton affinity, and partial atomic charges on donor and acceptor atoms. Steric and electronic interactions were further explored by visualizing HOMO-LUMO orbital overlaps.

2.2 Theory

2.2.1 Lewis acid and base

- **Lewis acid** is an electron pair acceptor. It possesses an empty orbital, often resulting from an electron-deficient central atom or a positive charge, which enables it to accept a pair of electrons from another species. This makes Lewis acids Electrophilic in nature, and they commonly have incomplete octets as in 13th Boron and Aluminum complexes. They have low energy LUMO orbital to accept electrons from HOMO of base.
- **Lewis base** is an electron pair donor. It has at least one lone pair of electrons available for donation, enabling it to form a coordinate covalent bond with a Lewis acid. Lewis bases are thus Nucleophilic, and typically feature atoms such as nitrogen, oxygen, or phosphorus with non-bonding electron pairs. They have high energy HOMO to donate electron to acids.
- **Acid–Base adduct** is a chemical compound formed when a Lewis base donates a lone pair of electrons to a Lewis acid, resulting in a coordinate covalent bond.

2.2.2 Binding Energy

Binding Energy (ΔE_{bind}) is the energy difference between the adduct and the sum of the energies of the separate acid and base. For a gas-phase reaction between a Lewis acid and a Lewis base, The binding energy is given by:

$$\mathrm{A(g)} + : \mathrm{B(g)} \rightarrow \mathrm{A{:}B(g)}$$

$$\Delta E_{bind} = E_{A:B} - (E_A \ + \ E_B)$$

$$If\ \Delta E_{bind} < 0 \Rightarrow Exothermic\ and\ Thermodynamically\ favourable$$

$$If\ \Delta E_{bind} > 0 \Rightarrow Endothermic\ and\ Thermodynamically\ unfavourable$$

2.2.3 Factors Affecting Binding Energy

A negative binding energy doesn't guarantee Lewis acid–base adduct formation, as factors like kinetics, sterics, and acid–base compatibility (e.g., hard–hard interactions) also influence binding. Here, we explore the key factors affecting adduct stability and binding energy.

i). Energies of HOMO and LUMO

- A higher energy Highest Occupied Molecular Orbital(HOMO), closer to 0 eV or less negative means more willing to donate electrons. Therefore, a base with a high HOMO is considered a stronger Lewis base.
- A lower energy Lowest unoccupied Molecular Orbital (LUMO), more negative means more favorable to accept electrons. Therefore, an acid with a low LUMO is considered a stronger Lewis acid.

ii). Hardness of acid and base The Hardness is the concept which measures the strength of lewis acid and base and classify them as hard and soft. The hardness (η) scale used here is Parr–Pearson Hardness (or Density Functional Theory-DFT Global Hardness) can be formulated as:

$$\eta = \frac{E_{\text{LUMO}} - E_{\text{HOMO}}}{2}$$

Large $\eta \Rightarrow$ Soft acid and base $\Rightarrow$ Species which resist change in electron density and less reactive.

Small $\eta \Rightarrow$ Hard acid and base $\Rightarrow$ Species which more polarisable and more reactive.

iii). Reorganizational Energy of idealised acid: It is the energy required to change the geometry of the acid from trigonal planar to the idealized pyramidal structure. The geometry change happens when an acid interact with base during the formation of adduct.

$$E_{reorg} = E_{planar} - E_{idealised\ pyramidal}$$

iv). Reorganizational Energy of acid and base in adduct The base is removed from the adduct, and the energy of the acid in the geometry it adopts within the adduct, E^A_{AB}, is calculated. The **reorganizational energy of the acid** is then given by $E^A_{AB} - E^A_0$, where E^A_0 is the energy of the free acid at its own optimized geometry.

Similarly, the acid is removed from the adduct, and the energy of the base in the geometry it adopts within the adduct, E^B_{AB}, is calculated. The **reorganizational energy of the base** is given by $E^B_{AB} - E^B_0$, where E^B_0 is the energy of the free base at its optimized geometry.

v). Vertical Electron Attachment Energy (VEAE) of acid It is the energy change when an electron is added to a molecule without allowing the molecule to change its geometry. The geometry of acid remains same for both the structures, either planar or idealized pyramidal.

$$E_{VEAE} = E_{pyamidal\ MX_3} - E_{pyamidal\ MX_3^-}$$

vi). Proton affinity of base It is a measure of the tendency of a base to accept a proton (H^+) in the gas phase. It is proportional with the basicity of a molecule.

$$\mathrm{B\ (g) + H^+ \rightarrow BH^+\ (g)} \qquad \mathrm{PA} = -\Delta H$$

$$\Delta H_{PA} = (\Delta H_{Base} + \Delta H_{H^+}) - \Delta H_{(Base-H)^+} \simeq \Delta H_{Base} - \Delta H_{(Base-H)^+}$$

vii). Charge transfer on adduct formation It is the extent of net Mulliken charge transfer from base to acid in an adduct. It is the charge difference on the donor atom in its isolated state and in adduct. It can be formulated as:

$$\Delta Q_{BA} = q_{isolated} \ - \ q_{adduct}$$

viii). Change in the M–X bond distance in the acid

$$\Delta r_{MX} = \text{M-X bond length of acid in adduct} - \text{M-X bond length of free acid}$$

ix). X–M–X bond angle in the acid of the adduct The $\angle XMX$ is the angle between the central and the substituent atoms. In a free acid with idealized pyramidal geometry, the angle is $109°50''$. But due to adduct formation, the $\angle XMX$ changes and it can be observed in table 7.

x). Electrostatic and steric interactions Steric and electronic interactions refer to the repulsion caused by bulky groups surrounding the acceptor and donor centers of a Lewis acid or base. These interactions can hinder the close approach required for effective orbital overlap and reduce or prevent adduct formation. Even when the electronic properties like hardness, HOMO & LUMO energy levels favor adduct formation, steric hindrance can distort geometry and lower the stability or block the interaction entirely. This can be observed for 2,6-dimethylpyridine–BCl_3 adduct in table 7, which despite involving one of the strongest Lewis acid-base pairs, exhibits lower binding energy compared to complexes such as NH_3-$AlCl_3$ and 3,5-dimethylpyridine–BCl_3. The reduced stability is attributed to steric hindrance from the methyl groups at the 2' and 6' positions of the pyridine ring, which prevents effective interaction with the Lewis acid.

Unit Conversion:

$$1\ Hatree = 1\ a.u. = 27.2114\ eV = 627.509\ kcal/mol = 2625.50\ kJ/mol$$

2.3 Observation

Molecule	E_{HOMO} **(eV)**	E_{LUMO} **(eV)**	**Hardness (eV)**	ΔE_{EA^a} **(kcal/-mol)**	$\Delta E^b_{\mathrm{reorg}}$ **(kcal/-mol)**	Q^c_{AA} (statC)
$\mathbf{BH_3}$	-12.87	1.15	7.01	8.29	24.37	0.03
$\mathbf{BF_3}$	-16.96	0.47	8.72	-5.65	46.18	1.19
$\mathbf{BCl_3}$	-12.85	-1.39	5.73	-63.76	40.66	0.31
$\mathbf{AlH_3}$	-11.28	-0.02	5.63	-13.91	20.69	0.76
$\mathbf{AlCl_3}$	-12.79	-2.29	5.25	-68.45	21.12	1.39

Table 5: Calculated Properties of the Idealized Pyramidal Boron and Aluminum Hydrides and Halides using *HF 3-21G basis set*

Lewis base	E_{HOMO} **(eV)**	E_{LUMO} **(eV)**	**Hardness (eV)**	**Proton Affinity (PA) (kcal/-mol)**	Q^d_{DA} (statC)
$\mathbf{NF_3}$	-15.01	6.02	10.52	126.24	0.65
$\mathbf{PH_3}$	-10.36	4.93	7.64	184.27	0.09
$\mathbf{NH_3}$	-10.59	7.49	9.04	226.94	-0.88
Pyridine	-9.69	3.54	6.61	241.22	-0.67
3,5-Dimethylpyridine	-9.13	3.56	6.34	246.06	-0.68
$\mathbf{N(CH_3)_3}$	-9.08	6.92	8.00	248.03	-0.68
2,6-Dimethylpyridine	-9.08	3.62	6.35	250.04	-0.75

Table 6: Calculated Properties of the Lewis Bases using *HF 3-21G basis set*

[a] Vertical electron attachment energy that occurs when an electron is added to the neutral MX_3 molecule at the idealized pyramidal geometry.
[b] The re-organizational energy necessary to change the geometry of the acid from trigonal planar to the idealized pyramidal structure.
[c] The partial charge on the acceptor atom.
[d] Partial charge on the donor atom

Adduct	ΔBE^a (kcal mol^{-1})	r^b_{MN} (Å)	r^c_{MX} (Å)	$\angle XMX^d$ (°)	$E^e_{\text{reorg acid}}$ (kcal mol^{-1})	$E^f_{\text{reorg base}}$ (kcal mol^{-1})	ΔQ^g_{BA}	ΔQ^h_{DA}	Q^i_{AA}
H_3N:$AlCl_3$	-66.98	1.98	0.04	116.50	6.61	0.54	0.25	-1.01	1.40
H_3N:BCl_3	-49.14	1.63	0.10	113.50	24.98	0.40	0.36	-0.90	0.27
H_3N:AlH_3	-42.88	2.05	0.02	116.86	5.42	0.33	0.19	-0.96	0.74
H_3N:BF_3	-42.09	1.69	0.05	114.13	24.11	0.20	0.20	-0.98	1.19
H_3N:BH_3	-35.65	1.71	0.02	113.95	13.59	0.22	0.26	-0.86	0.00
H_3P:$AlCl_3$	-27.27	2.54	0.04	116.74	6.11	2.56	0.14	-0.04	1.43
H_3P:AlH_3	-12.69	2.72	0.01	118.67	2.23	0.85	0.05	0.00	0.80
H_3P:BH_3	-8.39	2.21	0.01	116.80	6.96	1.07	0.27	0.24	-0.18
H_3P:BCl_3	-7.73	2.06	0.10	113.85	23.76	4.33	0.66	0.49	-0.16
H_3P:BF_3	-5.70	3.03	0.00	119.74	0.90	0.17	0.01	0.02	1.20
3,5-Dimethyl-pyridine:BCl_3	-47.92	1.61	0.12	111.07	33.72	0.65	0.36	-0.96	0.35
pyridine:BCl_3	-46.68	1.61	0.12	111.24	33.01	0.69	0.35	-0.94	0.35
2,6-Dimethyl-pyridine:BCl_3	-28.66	1.62	0.13	111.49	42.20	10.83	-0.36	-1.00	0.35

Table 7: Calculated properties of Ammonia and Phosphine adducts of Lewis acid using *HF 3-21G basis set.*

[a] Binding energy.
[b] Donor atom-acceptor atom bond distance in the adduct.
[c] Change in the M-X bond distance in the acid of the adduct.
[d] Value of the X-M-X bond angle in the acid of the adduct.
[e] Re-organizational energy of the acid upon adduct formation as a result of a change in the structure.
[f] Re-organizational energy of the acid upon adduct formation as a result of a change in the structure.
[g] Net Mulliken Charge transfer from the base to the acid upon adduct formation.
[h] Charge on the donor atom of the base in the adduct.
[i] Charge on the acceptor atom of the acid in the adduct.

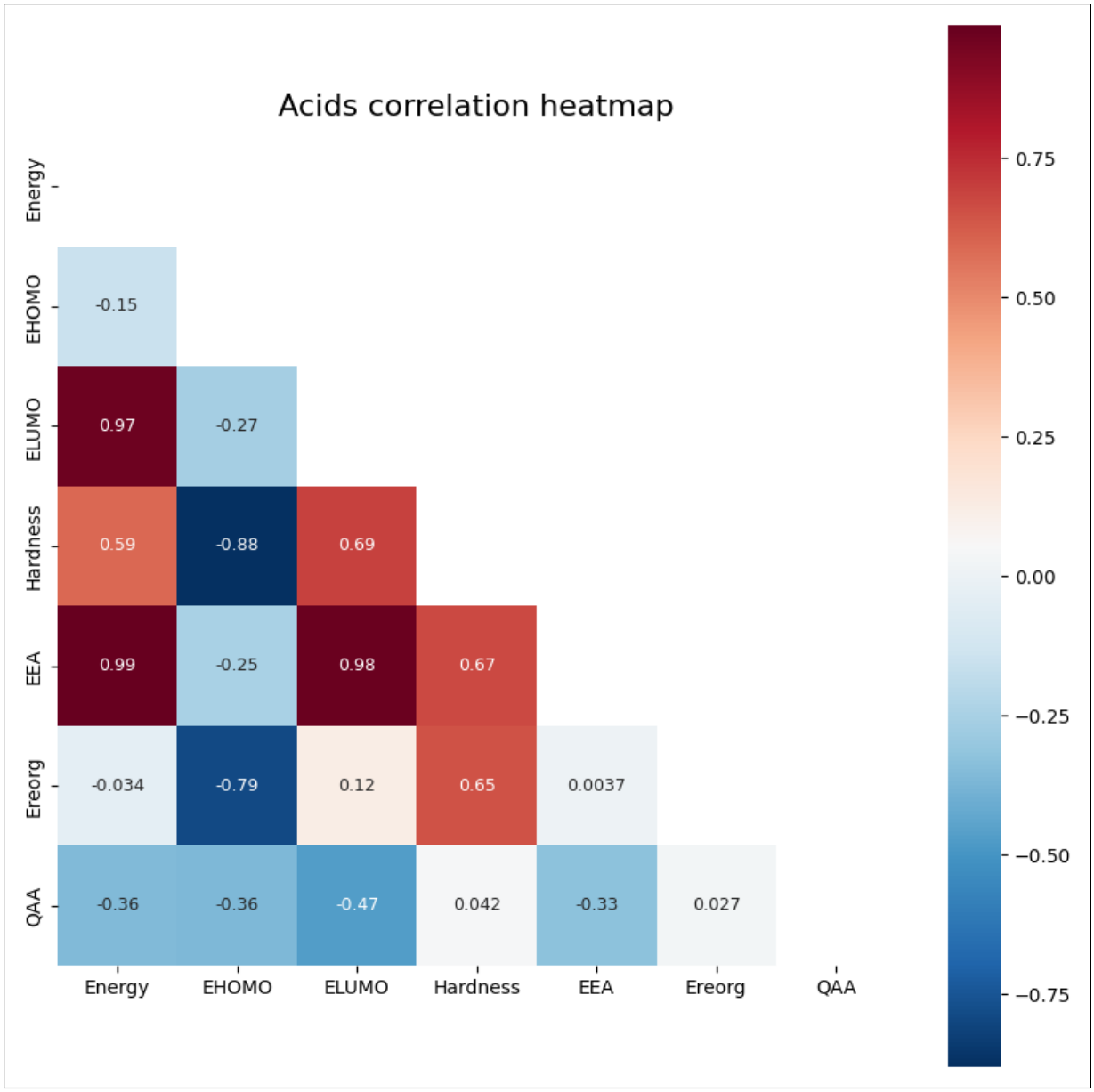


Figure 2: Heatmap with corelation coefficient showing relation between different quantitative variables of Acids

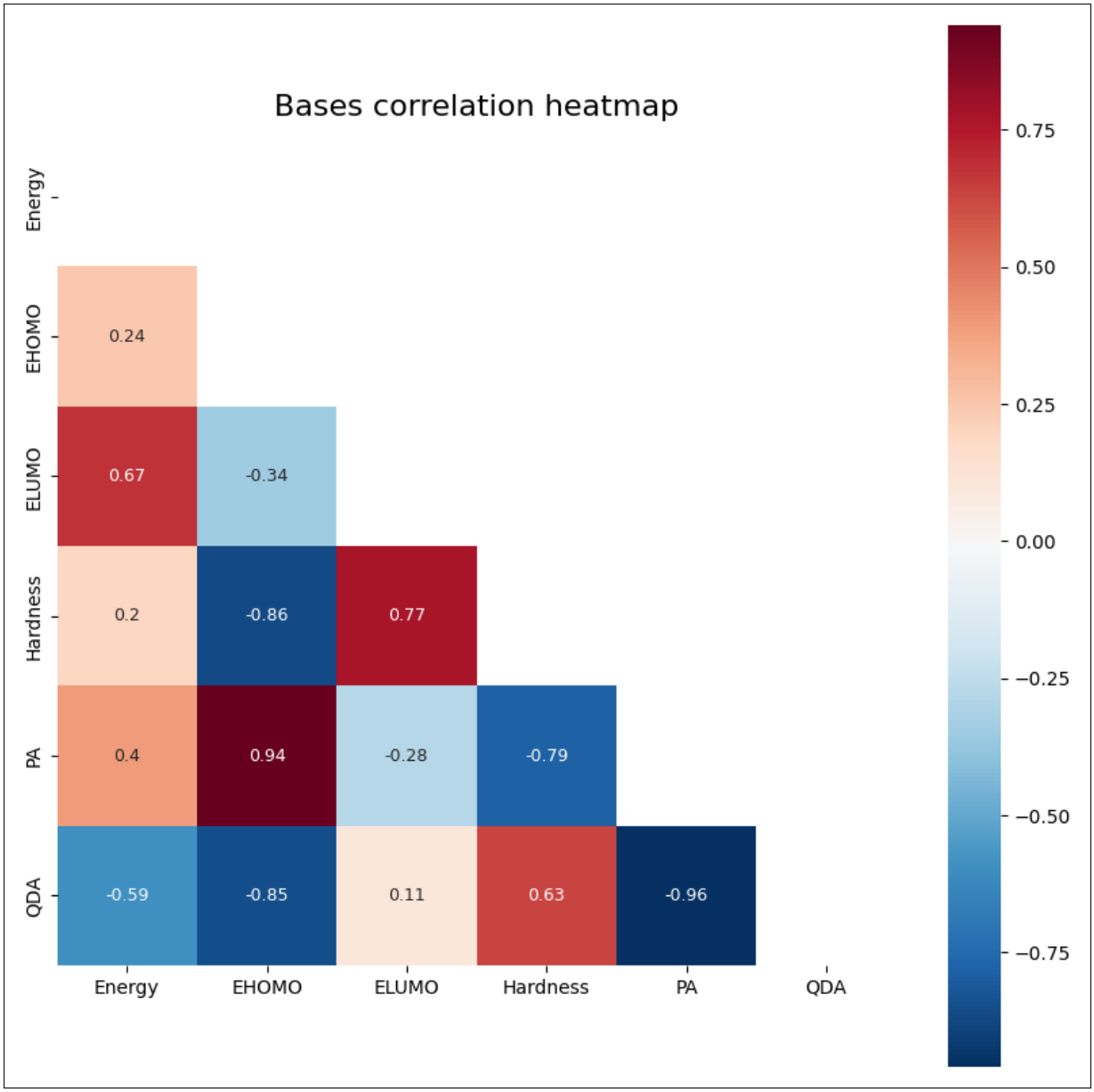


Figure 3: Heatmap with corelation coefficient showing relation between different quantitative variables of Bases

2.4 Results

2.4.1 Strength of acid, base and adducts

From the observation of hardness value of acids from table 5, the descending order of the strength of acid can be given as:

$$AlCl_3 > AlH_3 > BCl_3 > BH_3 > BF_3$$

From the observation of hardness value of acids from table 6, the descending order of the strength of base can be given as:

$$3,5{-}dimethylpyridine \simeq 2,6{-}dimethylpyridine > pyridine > PH_3 > N(CH_3)_3 > NH_3 > NF_3$$

From observation of the binding energy value of the Lewis acid ammonia and phosphorus adducts of table 7, the descending order of the strength of the adducts can be given as:

NH_3–$AlCl_3 > NH_3$–$BCl_3 > 3,5$-dimethylpyridine–$BCl_3 >$ pyridine–$BCl_3 >$
NH_3–$AlH_3 > NH_3$–$BF_3 > NH_3$–$BH_3 > PH_3$–$AlCl_3 >$
$2,6$-dimethylpyridine–$BCl_3 > PH_3$–$AlH_3 > PH_3$–$BH_3 > PH_3$–$BCl_3 > PH_3$–BF_3

2.4.2 Correlation Heatmap

A correlation heatmap is a data visualization technique that uses color to represent Pearson's correlation coefficient (r) in a 2D matrix. The value of the correlation coefficient ranges from 1 to -1.

$r = +1 \Rightarrow$ Perfect positive linear correlation (all points lie exactly on a line with positive slope)
$r \in (0, 1) \Rightarrow$ Weak to strong positive linear correlation with deviations from straight line
$r = 0 \Rightarrow$ No linear or non-linear correlation
$r \in (-1, 0) \Rightarrow$ Weak to strong negative linear correlation with deviations from straight line
$r = -1 \Rightarrow$ Perfect negative linear correlation (all points lie exactly on a line with negative slope)

From the acid correlation heat map shown in fig. 2, we can visualize the relationship between different factors affecting the hardness of acid. We can write the following conclusion from it.

$$Strength \propto \frac{1}{Hardness} \propto \frac{1}{E_{LUMO}} \propto E_{HOMO} \propto \frac{1}{E_{EA}} \propto \frac{1}{E_{reorg}} \propto \frac{1}{Energy}$$

From the base correlation heat map shown in fig. 3, we can visualize the relationship between different factors affecting the hardness of base. We can write the following conclusion from it.

$$Strength \propto \frac{1}{Hardness} \propto \frac{1}{E_{LUMO}} \propto E_{HOMO} \propto \Delta E_{PA} \propto \frac{1}{Q_{DA}}$$

From the adduct correlation heat map shown in **??**, we can visualize the relationship between different factors that affect the hardness of the Ammonia and Phosphine adducts of Lewis acid. From this we can draw the following conclusion.

$$\Delta E_{bind} \propto \frac{1}{E_{base}} \propto Q_{DA} \propto r_{MN} \propto \angle XMX \propto E_{acid}$$

$$r_{MX} \propto \Delta E_{reorg\ acid} \propto \frac{1}{\angle XMX}$$

$$r_{MN} \propto \angle XMX \propto Q_{DA} \propto \frac{1}{\Delta E_{reorg\ base}} \propto \frac{1}{r_{MX}}$$

2.5 Discussion and Conclusion

This study demonstrated that *ab-initio single point Hatree Fock HF/3-21G* calculations effectively capture trends in Lewis acid-base adducts binding energies and related electronic properties. The Key parameters such as E_{LUMO}, E_{HOMO}, hardness, and proton affinity were consistent with higher-level data from literature (Anderson, 2000), confirming the method's reliability for qualitative analysis.

Overall, this work provides a foundation for future studies involving more complex systems or higher levels of theory. The ability to predict and interpret acid-base behavior computationally is a powerful tool for guiding experimental design and advancing theoretical understanding in coordination chemistry, catalysis, and materials science.

. . . . . .

3 Project: III - Machine Learning for Chemists: Using Python Notebooks for Visualization, Data Processing, Analysis, and Modeling of wine dataset

[3]

3.1 Introduction

The integration of automation and Internet of Things (IoT) technologies in laboratories has revolutionized chemical research by enabling the collection of large, complex datasets involving multiple physicochemical variables. With this shift, data analysis and scientific computing—particularly machine learning—have become vital tools for extracting insights and building predictive models.

This study focuses on developing foundational machine learning skills using Python, with an emphasis on real-world data analysis. Using interactive platforms such as Jupyter Notebook and Google Colab, we explored a dataset comprising the physicochemical properties of red and white wines. We acquired basic knowledge of essential Python libraries used in machine learning and applied various techniques for data visualization, pre-processing, analysis, and modeling of the wine dataset.

3.2 Basic Definitions

- **Dataset:** It is a structured collection of data organized in tabular form, where each row represents a single observation or record, and each column corresponds to a specific variable or feature.
- **Dataframe:** It is a two-dimensional, size-mutable, and heterogeneous data structure in Python provided by the Pandas library

Table 8: Dataset Description from Wine Dataset Containing Sample Quantities of Each Wine Type, Wine Physicochemical Attributes, and Their Corresponding Units

input variables

No.	**Features**	**Units**
1	fixed acidity	g(tartaric acid)/dm^3
2	volatile acidity	g(acetic acid)/dm^3
3	citric acid	g/dm^3
4	residual sugar	g/dm^3
5	chlorides	g(sodium chloride)/dm^3
6	free sulfur dioxide	mg/dm^3
7	total sulfur dioxide	mg/dm^3
8	density	g/cm^3
9	pH	–
10	sulfates	g(potassium sulfate)/dm^3
11	alcohol	vol %

output variable

No.	**Labels**	**Units**
12	quality	–

Samples	1599	red wine
	4898	white wine

As shown in table 8, both the red and white wine datasets contain 12 variables, out of which 11 input variables are derived from the chemical analysis of wine, and one output

Table 9: Python Libraries Used in the Notebooks to Analyze and Plot the Data

Library Name	**Description**	**Website**
Matplotlib	Plotting tools	https://matplotlib.org/
NumPy	Linear algebra and fast math library	https://numpy.org/
Pandas	Handling and preprocessing of large data sets	https://pandas.pydata.org/
Pingouin	Common statistical tests	https://pingouin-stats.org/
Seaborn	Advanced plotting functions	https://seaborn.pydata.org/
Plotly	Interactive plots and advanced plotting functions	https://plotly.com/python/
Scikit-Learn	Scientific computing and machine learning tools	https://scikit-learn.org/

variable "quality" which reflects the effectiveness of the wine based on these inputs.

We utilized all the Python libraries listed in table 9, along with Ipywidgets, to create interactive and advanced 3D visualizations.

3.2.1 Importing wine datasets

```
red_df = pd.read_csv("https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv", delimiter=";")
white_df = pd.read_csv("https://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-white.csv", delimiter=";")
```

3.3 Data Visualization using statistical plots

3.3.1 Histogram

```
sns.set()
red_df.hist(figsize=(12,12), color='red')
plt.show()
```

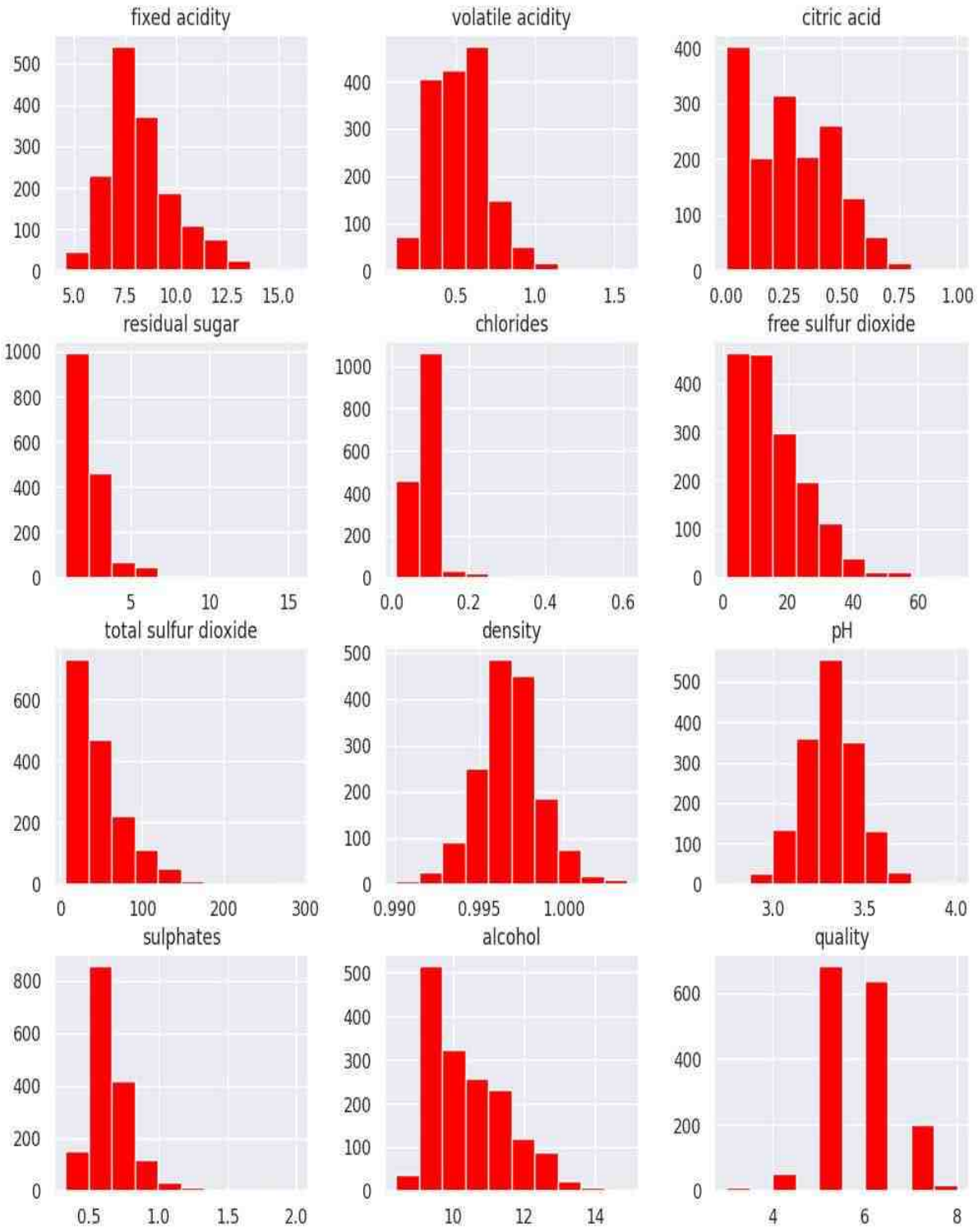


Figure 5: Histogram plot of Red wine dataframe

```
sns.set()
white.hist(figsize=(12,12), color='red')
plt.show()
```

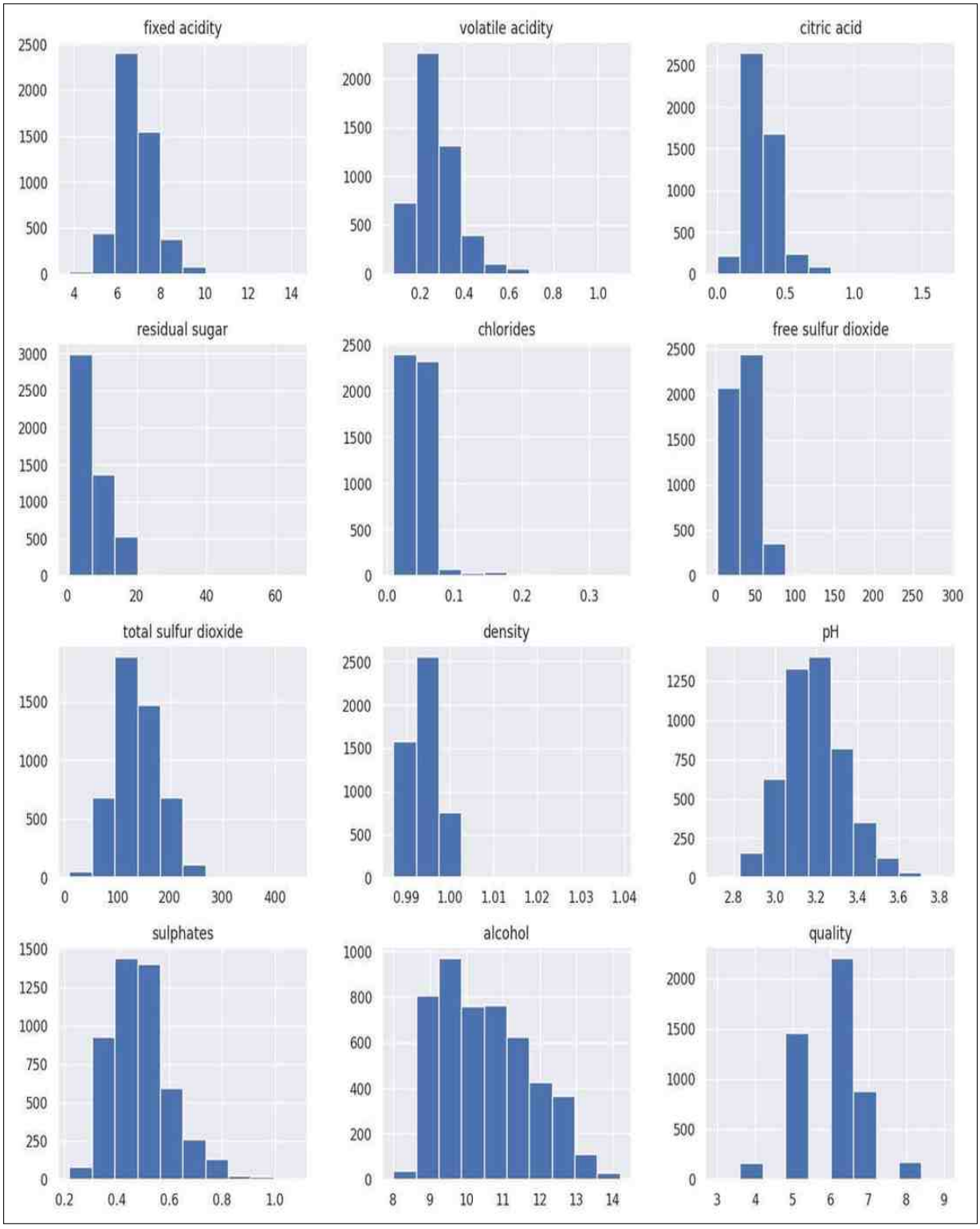


Figure 6: Histogram plot of White wine dataframe

3.3.2 Box plot

```
log_box = sns.boxplot(data=red_df, orient="h", palette="rainbow")
log_box.set_xscale("symlog")
log_box.axis(xmin=0, xmax=300)
```

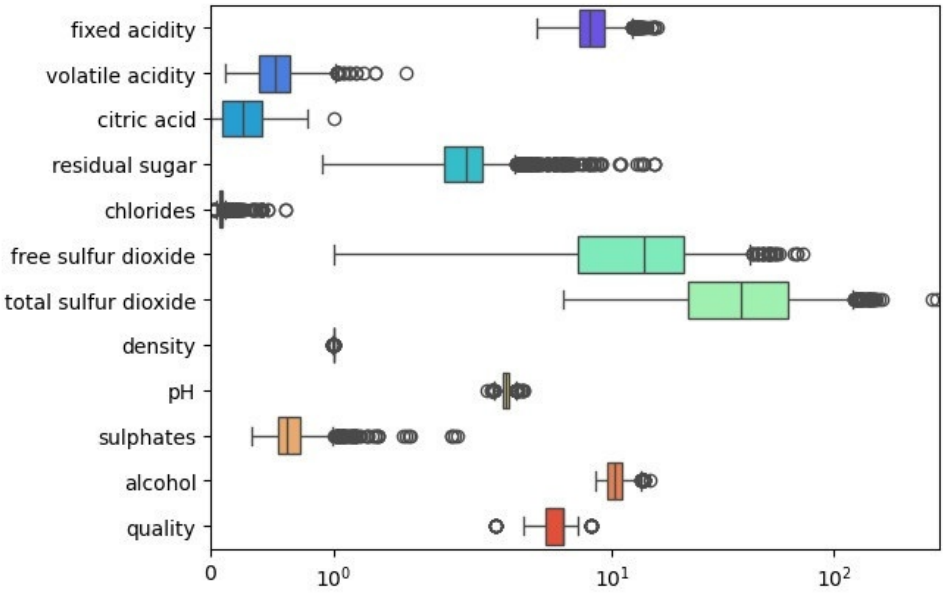


Figure 7: log scale Box plot of Red wine dataset

3.3.3 Violin plot

```
sns.violinplot(x="quality", y="alcohol", data=red_df, hue="
    quality" ,palette="rainbow")
plt.show()
```

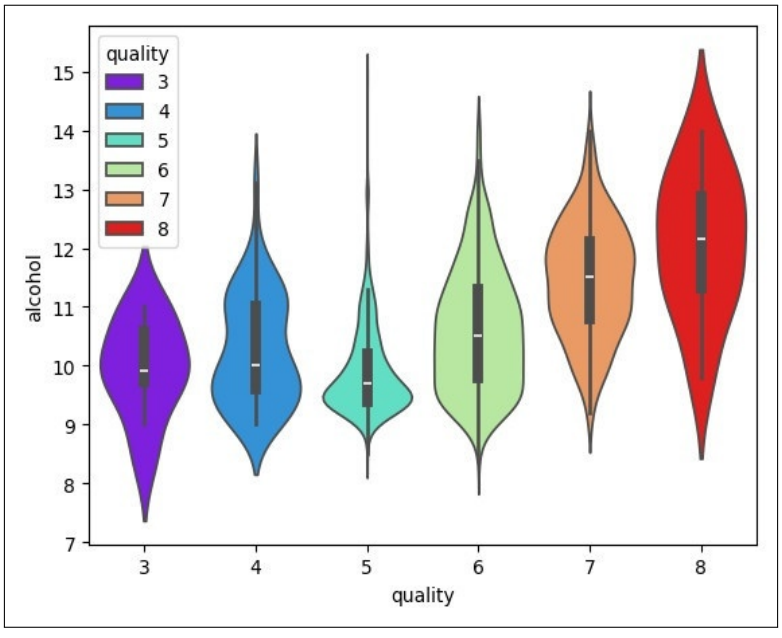


Figure 8: Violin plot between "quality" and "alcohol content" in red wine

3.3.4 Heatmap

```
red_correlation = red_df.corr(method='pearson')
fig=plt.gcf()
fig.set_size_inches(12,12)
sns.heatmap(red_correlation, annot=True,square=True, cmap="
    coolwarm")
```

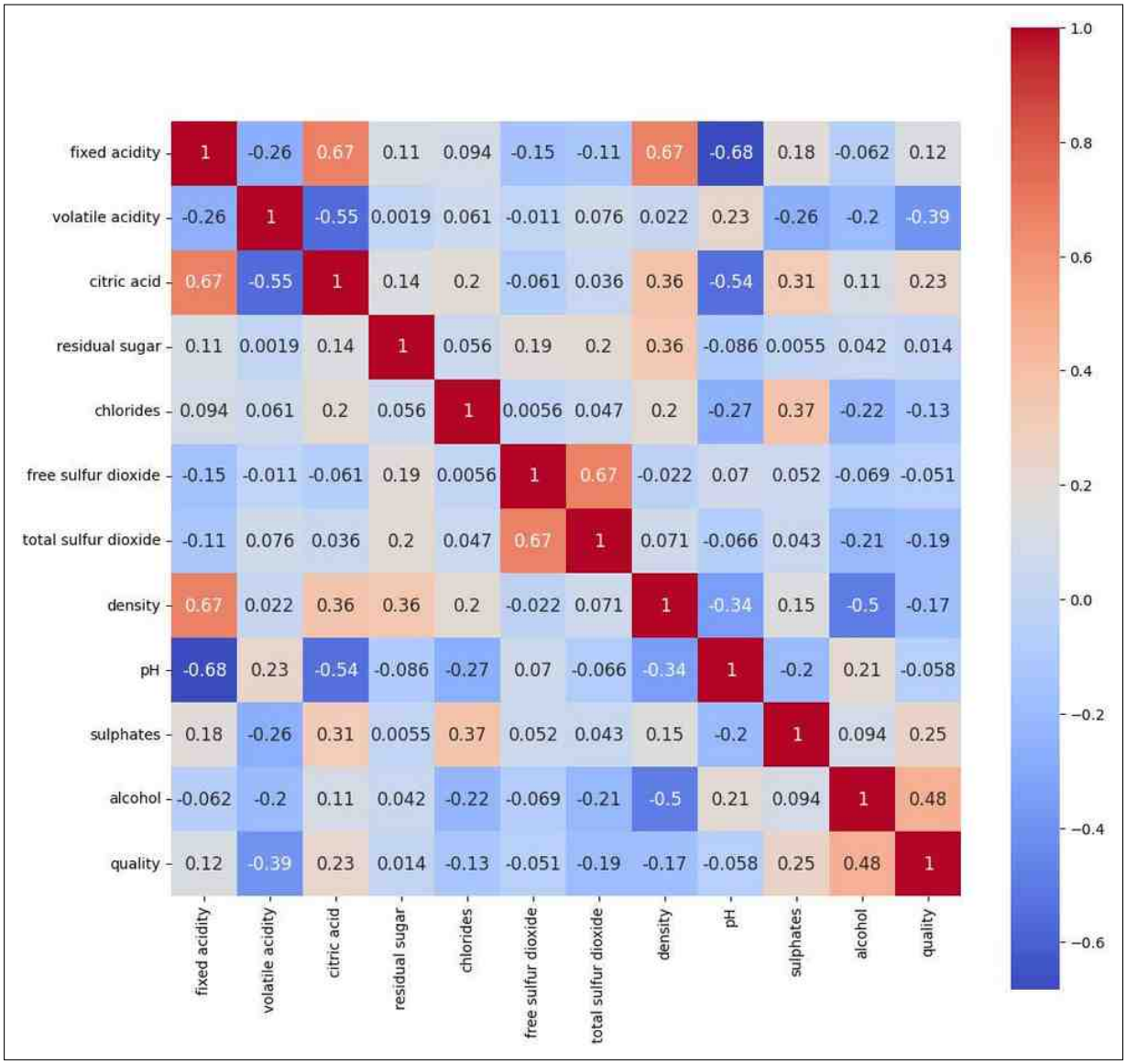


Figure 9: Correlation Heatmap of White wine dataset

```
white_correlation = white_df.corr(method='pearson')
fig=plt.gcf()
fig.set_size_inches(12,12)
sns.heatmap(white_correlation, annot=True, square=True, cmap="
    coolwarm")
```

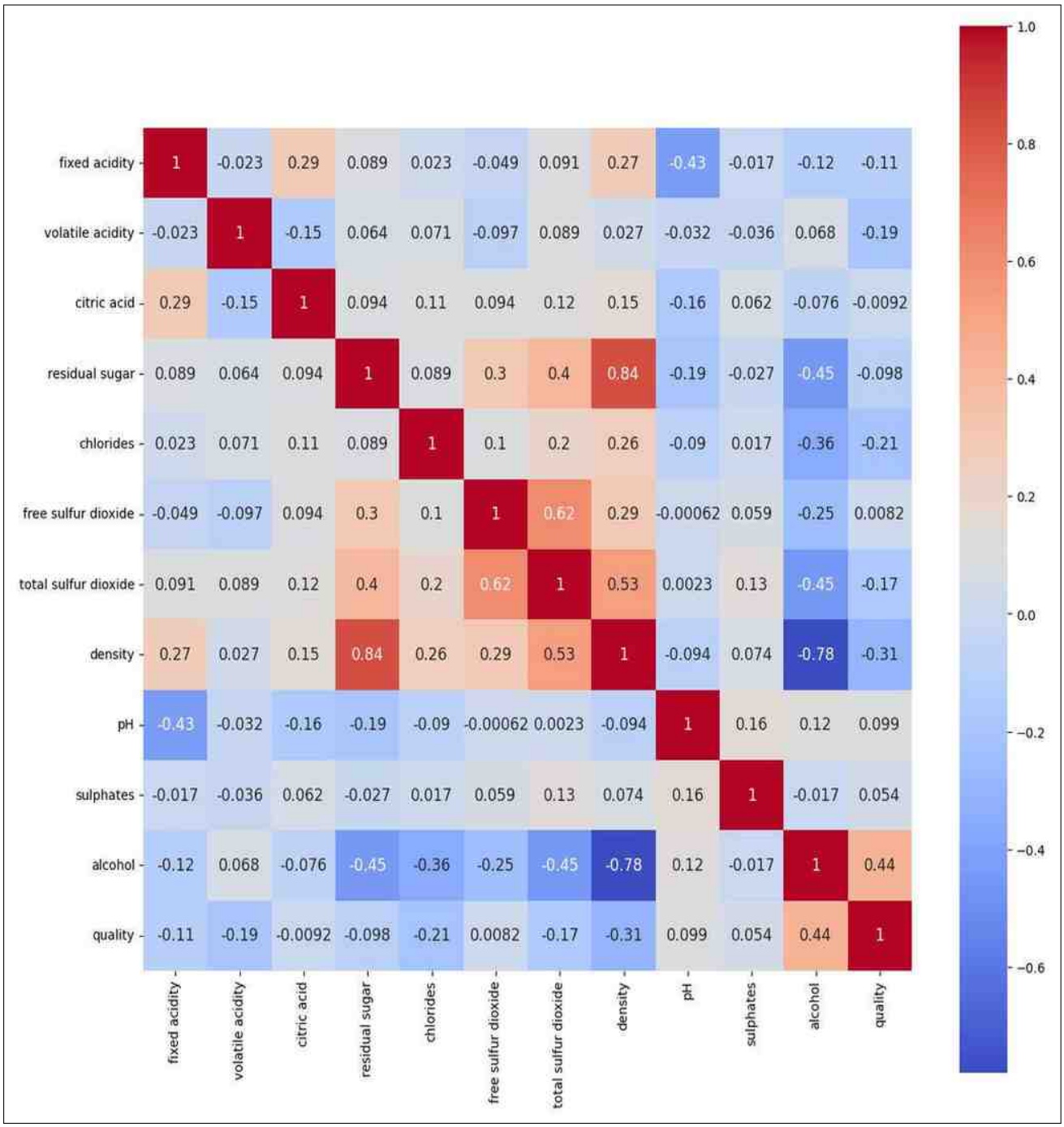


Figure 10: Correlation Heatmap of White wine dataset

3.3.5 Pair plot

```
sns.pairplot(red_df[['fixed acidity','volatile acidity','citric
    acid', 'pH']], corner=False)
#corner=True hides the upper portion of the matrix
```

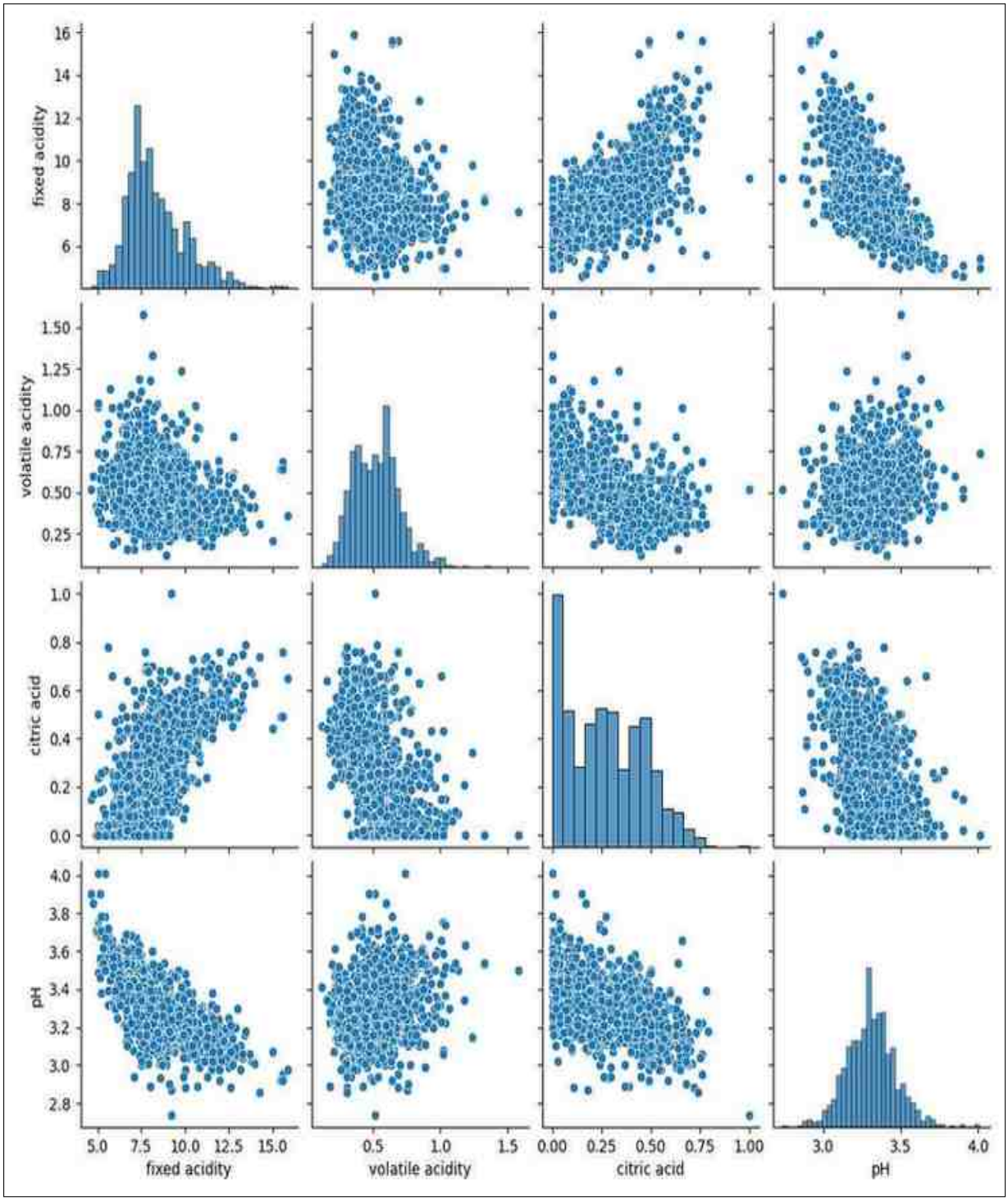


Figure 11: Pair scatterplot between "fixed acidity", "volatile acidity", "citric acid", "pH" of red wine dataset

3.4 Statistics in pandas

3.4.1 Descriptive Statistics functions

```
red_df.info() #Returns datatype and data range
red_df.head(10) # Returns first 10 row values
red_df.tail(10) #Returns last 10 row values
red_df.max(axis=0) #List out max value of all column variables
red_df["pH"].min() #Returns min value of "pH" column only
red_df.describe() #Returns count,mean,std,min,max & percentile
red_df.groupby(["quality"])[["alcohol"]].describe()
red_df.std(axis=0) #Returns standard deviation of all variables
    in column
```

Listing 1: Statistics functions integrated with pandas

3.4.2 Confidence interval

A confidence interval is a range of values derived from a sample data that is believed to contain the true value of an unknown population parameter with a specified probability. For example, if you calculate a 95% confidence interval for a population mean, it means you are 95% confident that the true mean lies within that interval based on the method and sample used. It can be mathematically formulated as:

$$CI \;=\; Mean \pm z \times SE \quad where\;,\; SE = \frac{SD}{\sqrt{n}}$$

z is the z-value, when calculating a confidence interval, is the value of the z (normal distribution) statistic for a certain confidence level. Common values of z for different confidence intervals are:

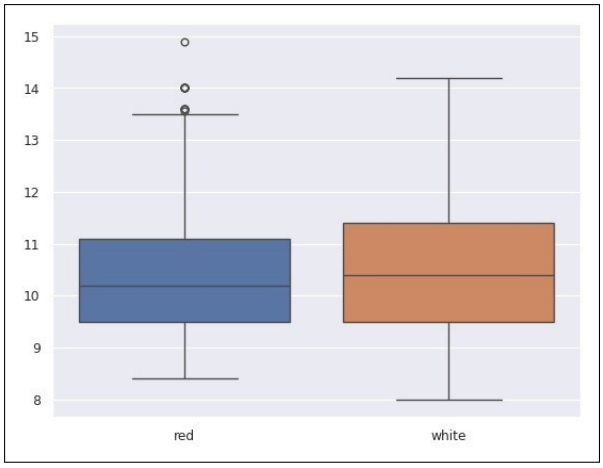
$$\text{Confidence Level: } 90\%\ (Z = 1.645), \quad 95\% \Rightarrow\ (Z = 1.96), \quad 99\% \Rightarrow\ (Z = 2.575)$$

SE is the Standard error of the mean. It indicates the uncertainty around the estimate of the mean.
n is the sample size

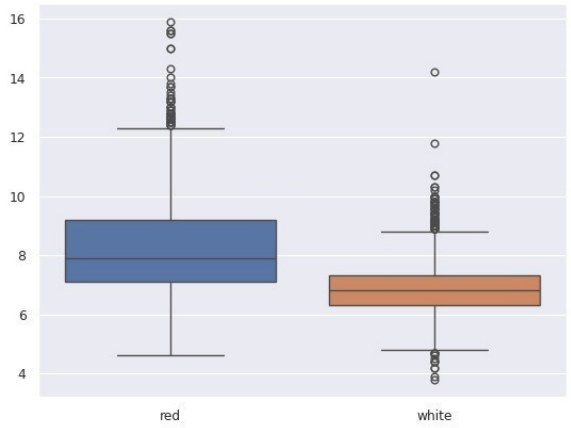
3.5 Hypothesis tests

When comparing two samples (or analytical methods) it's important to analyze if their means and/or variances are equal. In order to decide if the difference in their means and variances can be accounted to random variations, a statistical test can be used. These tests are also known as significance tests.

```
# Comparing only fixed acidity of red and white wine
fa_df = pd.concat([red_df["fixed acidity"],white_df["fixed
    acidity"]], ignore_index=True, axis=1)
fa_df.columns = ['red', 'white']
sns.boxplot(data = fa_df)
sns.violinplot(data = fa_df)
```



(a) Alcohol content box plot



(b) Fixed acidity box plot

Figure 12: Hypothesis test plot between red and white wine

Conclusion from hypothesis test: We notice that the distribution of alcohol content is very similar for red and white wines, while the distribution of fixed acidity is significantly different. Let's see if we can statistically demonstrate this observation.

3.6 Student's t-test

The Student's t-test is used to compare the mean of one or two normally distributed populations, preferably of equal size and variance by proposing an estimator of the likelihood of a Null Hypothesis. A distribution area related value called the "p-value" is then calculated. A very small p-value implies that Null Hypothesis is very unlikely. The usual criteria is to use a critical value of 0.05 for symmetrical distributions called the "level of significance"; if the calculated p-value is less than 0.05, then we conclude that the Null Hypothesis is very unlikely to be true.

3.6.1 One-sample test

In one-sample t-test we want to test the null hypothesis that the population mean $\bar{x}$ is equal to a specified value μ_0. First we will determine the mean of the fixed acidity in our dataset.

```
fa_df.mean()
Output:
red      8.319637
white    6.854788
```

Now that we know that the mean ($\bar{x}$) for red wine is 8.31, we will compare it to the "true value" of 8.33 (that might have come from the certificate of analysis of a standard reference material for example).

```
test1=pg.ttest(fa_df['red'],8.33)
test1

Output:
```

```
| Test Type                | T-test        |
| T-statistic              | -0.237999     |
| Degrees of Freedom (dof) | 1598          |
| Alternative Hypothesis   | Two-sided     |
| p-value                  | 0.811912      |
| 95% Confidence Interval  | [8.23, 8.41]  |
| Cohens-d                 | 0.005952      |
| Bayes Factor (BF10)      | 0.029         |
| Statistical Power        | 0.056506      |
```

As seen by the p-value (p-val= 0.81) the distribution mean could be 8.33. Let's perform the test again using a "true" white wine mean of 6.85.

```
test2=pg.ttest(fa_df['red'],6.85)
test2
Output:
| Test Type                | T-test                      |
| T-statistic              | 33.752939                   |
| Degrees of Freedom (dof) | 1598                        |
| Alternative Hypothesis   | Two-sided                   |
| p-value                  | 5.398298e-189               |
| 95% Confidence Interval  | [8.23, 8.41]                |
| Cohens-d                 | 0.844087                    |
| Bayes Factor (BF10)      | 8.726e184                  |
| Statistical Power        | NaN                         |
```

In this case, the p-value is $5.39e^{-189}$. This means it is not probable that 6.85 is the (μ_0) of the fixed acidity content for red wine.

3.6.2 Two-sample test

We can directly compare the fixed acidity values for both kinds of wine by using a two independent sample t-test (since the wines we are analyzing are independent). The independence of the samples is indicated with the parameter paired=False.

The correction parameter is used when both populations have different sample sizes, by setting it to "auto", it will automatically apply a correction when the sample sizes are unequal. This correction uses an equation called Welch-Satterthwaite's equation, that implies making complicated calculations involving the standard deviations.

```
fa_df["white"].dropna()
print(fa_df.mean())
test3=pg.ttest(fa_df['red'],fa_df['white'], paired=False,
   correction='auto')
test3
Output:
red      8.319637
white    6.854788

| T           | 32.422711     |
| dof         | 1848.947934   |
| alternative | two-sided     |
```

```
| p-val        | 5.668161e-183 |
| CI95%        | [1.38, 1.55]  |
| cohen-d      | 1.293371      |
| BF10         | 4.834e+209    |
| power        | 1.0           |
```

```
# Two sample t-test for alcohol content
print(al_df.mean())
test4=pg.ttest(al_df['red'],al_df['white'], paired=False,
   correction='auto')
test4
Output:
red      10.422983
white    10.514267

| T           | -2.859029       |
| dof         | 3100.474621     |
| alternative | two-sided       |
| p-val       | 0.004278        |
| CI95%       | [-0.15, -0.03]  |
| cohen-d     | 0.076571        |
| BF10        | 1.903           |
| power       | 0.757462        |
```

As the p-value is less than 0.05, we can say the difference in alcohol content is statistically significant. This points to a strong evidence against the null hypothesis, meaning we can safely say that both populations are different.

Upon inspection, we can see that the value for the degrees of freedom (DOF) is a decimal number. Given that the degrees of freedom should be (N-1), N being the sample size, one would expect N to be a whole number. As the sample size for red wine is smaller than the one for white wine (1599 vs 4898), the pingouin library estimated the DOF using the Welch–Satterthwaite equation. That's also why the DOF differs for "alcohol" and "fixed acidity", in both the sample size is the same but they have different standard deviations. If we use the parameter correction="auto" pengouin will use the W.S equation even if the sample size is the same for each column.

In case we needed to compare the results of a certain treatment on a material system or population (where the two groups of measure would be dependent), the correct way to perform a two sample t-test would be to indicate that the samples are dependent; this is usually called a paired t-test.

3.7 Equality of Variances (Homoscedasticity)

A common assumption for many tests is that the variance for the different sets must be the same. So it's always handy to have a tool that can check this assumption before processing. You must have heard of the F-test. It's the simplest test for this job, however there are many more robust and powerful tests, and Pingouin doesn't offer an F-test function in his repertory. However another common test is Levene's test, which is more robust regarding lack of normality and more suitable on many situations (W statistic that follows an F distribution). Pingouin also offers another common test called Bartlett's test.

```
print('SD of "alcohol content": \n',al_df.std())
Output:
SD of "alcohol content":
 red       1.065668
white     1.230621
```

```
test5=pg.homoscedasticity(data=[red_df["alcohol"].to_numpy(),
   white_df["alcohol"].to_numpy()], alpha=0.05)
test5
Output:
levene test
| W           | 72.784614     |
| pval        | 1.780219e-17  |
| equalvar    | False         |
```

3.8 ANOVA

ANOVA (Analysis of Variance) is a statistical test used to determine whether the means of multiple groups are significantly different. It requires a categorical independent variable (E.g., wine quality) and a continuous dependent variable (E.g., residual sugar or alcohol content). ANOVA compares the within-group variance to the between-group variance and computes an F-statistic along with a p-value. A low p-value suggests that at least one group mean differs significantly. For example, using the wine dataset, we can compare the mean residual sugar across different quality levels (3 to 10). Pingouin provides a simple method to perform ANOVA directly on pandas DataFrames.

```
red_df.anova(dv="residual sugar", between="quality")
Output:
| Source      | quality    |
| ddof1       | 5          |
| ddof2       | 1593       |
| F           | 1.053374   |
| p-unc       | 0.384619   |
| np2         | 0.003295   |
```

Here **dv** means dependent variable, **between** ='quality' means that 'quality' is our independent (categorical) variable. We could safely conclude from this analysis, given the high p-value (¿0.05, here stated as "p-unc"), that the residual sugar will be the same regardless of the quality of the wine (or at least the mean of all the wines that were analyzed).

Let's analyze another variable, like the alcohol content, and let's see if we arrive to a different conclusion:

```
red_df.anova(dv="alcohol", between="quality")
Output:
| Source      | quality        |
| ddof1       | 5              |
| ddof2       | 1593           |
| F           | 115.854797     |
```

```
| p-unc       | 1.209895e-104 |
| np2         | 0.266667      |
```

A we can see, the p-value is too low, so we can say for sure that at least two values of alcohol content of wines of different qualities are indeed different.

3.9 Variable Reduction

A very useful way to visualize data with multiple variables are variable reduction techniques. Which reduce the multiple dimensions of the problem to just two or three, in order to make a plot that allows us to visualize the overall distribution of the variables. We will concatenate the datasets of red and white wine for the Purposes of visualization.

3.9.1 PCA (Principal Components Analysis)

The PCA algorithm defines a new set of coordinates (components) from a large dataset with multiple variables, and transforms the values into the new coordinates. These coordinates are defined in order to maximize the variance by some scalar projection. The components are arranged according to their original variance, that's why this technique is useful for reducing the dimensions of a dataset.

After loading the dataset, we import the Standard Scaler from `sklearn.preprocessing` and PCA function from `sklearn.decomposition`. Scaling the data is important as not every variable has the same range. So, we create the PCA object and define the number of components. Using `StandardScaler()` and `fit_transform()`, we can fit the new variables (components) and transform their values into the new variables. `StandardScaler()` does scaling the data. The `fit_transform()` module fits these new values to the data, and stores them, replacing the old values.

This is the most fastest variable reduction technique as it only uses basic algebra equation for finding Principle Components 1 & 2 and it will use them to plot a graph. But it can be use only for linear data. Since for non-linear data, the principle components will overlap with each other and the result is not consistent.

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

data_pca = df_wine.copy()
data_pca = data_pca.drop(labels = ['quality', 'hue'],axis = 1)
data_pca = StandardScaler().fit_transform(data_pca)
#Apply PCA on the transformed (scaled and centered) data:
pca = PCA(n_components=3)
pca_results = pca.fit_transform(data_pca)
pca_results

Output:
array([[-3.20599617,  0.41652332, -2.72223659],
   [-3.03905081,  1.10746213, -2.04695235],
       [-3.07189347,  0.87896444, -1.74257961],       ...,
       [ 0.5711325 , -0.72266165,  0.09146896],       [
          0.09005243, -3.54577991,  0.14119458],
```

```
        [ 0.51257566, -2.89104008,  0.73941657]])
```

```
pca_dataset = pd.DataFrame(data = pca_results, columns = ['
    component1', 'component2','component3'] )
pca_dataset['hue']=df_wine['hue']
plt.figure()
plt.figure(figsize=(6,6))
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.title('2 Component PCA')
sns.scatterplot(x = pca_dataset['component1'], y = pca_dataset['
    component2'], hue=pca_dataset['hue'],alpha=1,palette=["red", "
    blue"])
```

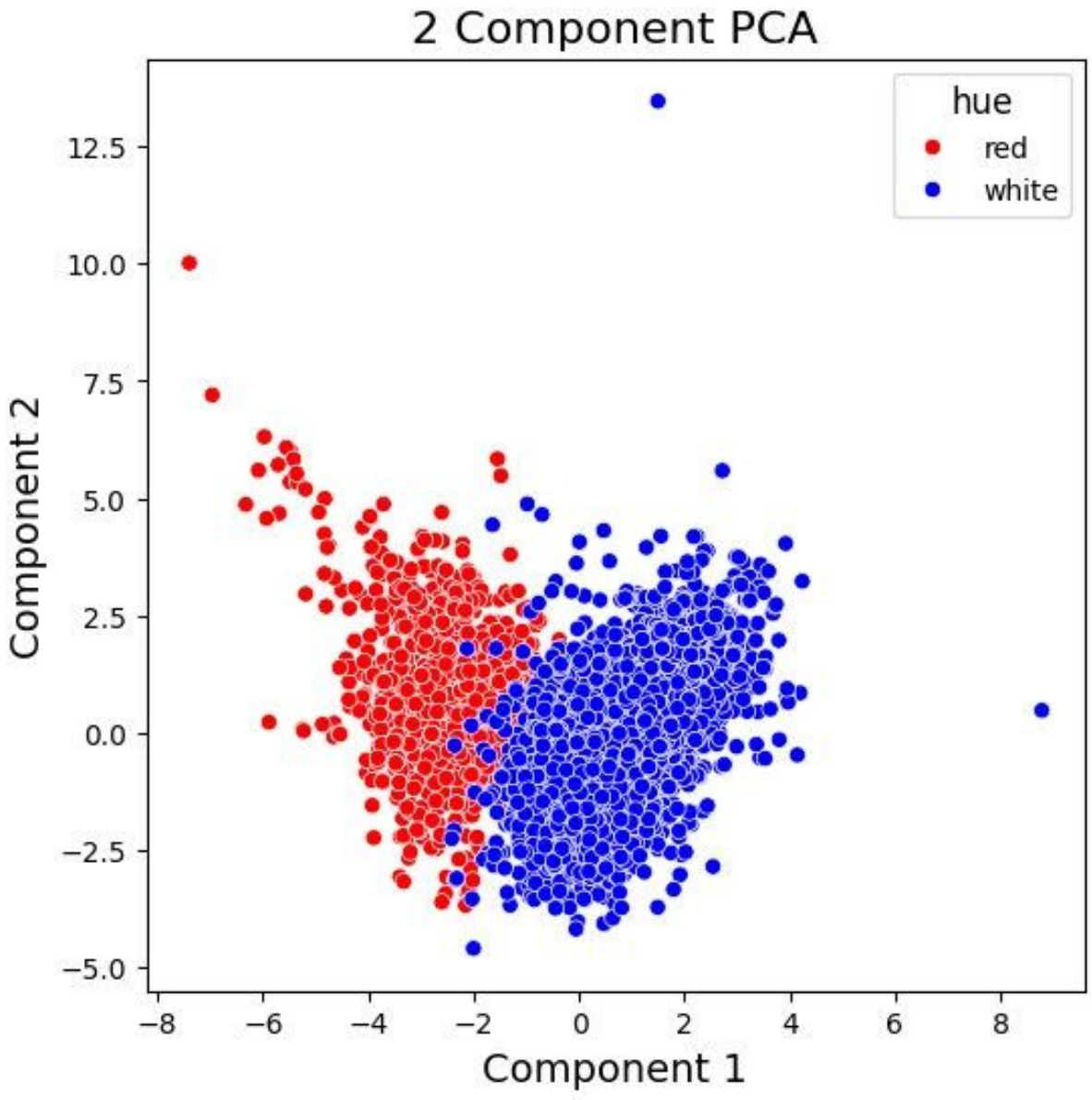


Figure 13: 2-dimensional PCA plot

3.9.2 t-SNE (t-distributed Stochastic Neighbor Embedding)

t-SNE is an algorithm that determines the similarity between pairs of points in a high dimensional space and replicates this similarity in a low dimensional space based on the probability distribution of the distances.

We can make this dimension reduction with the scikit learn module and then plot it with seaborn. First, we make a copy of the dataset, then, we drop the columns for quality and wine in this copy, as we won't be using them to make the t-SNE, but we will take the wine column into account for the plot, for the purpose of visualization.

To make the dimension reduction we have to create a t-SNE object and then use the `tsne.fit_transform()` function on it with our dataset copy.

```
from sklearn.manifold import TSNE
data_tsne = df_wine.copy()
data_tsne = data_tsne.drop(labels = ['quality', 'hue'],axis = 1)
data_tsne = StandardScaler().fit_transform(data_tsne)
tsne = TSNE(n_components=2, verbose=1, perplexity=40, n_iter=300)
tsne_results = tsne.fit_transform(data_tsne)
```

```
tsne_dataset = pd.DataFrame(data = tsne_results, columns = ['
    component1', 'component2'] )
tsne_dataset['hue']=df_wine['hue']
plt.figure()
plt.figure(figsize=(10,10))
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.title('2 Component TSNE')
sns.scatterplot(x = tsne_dataset['component1'], y = tsne_dataset[
    'component2'], hue=tsne_dataset['hue'],alpha=1, palette=["red"
    , "blue"])
plt.legend()
```

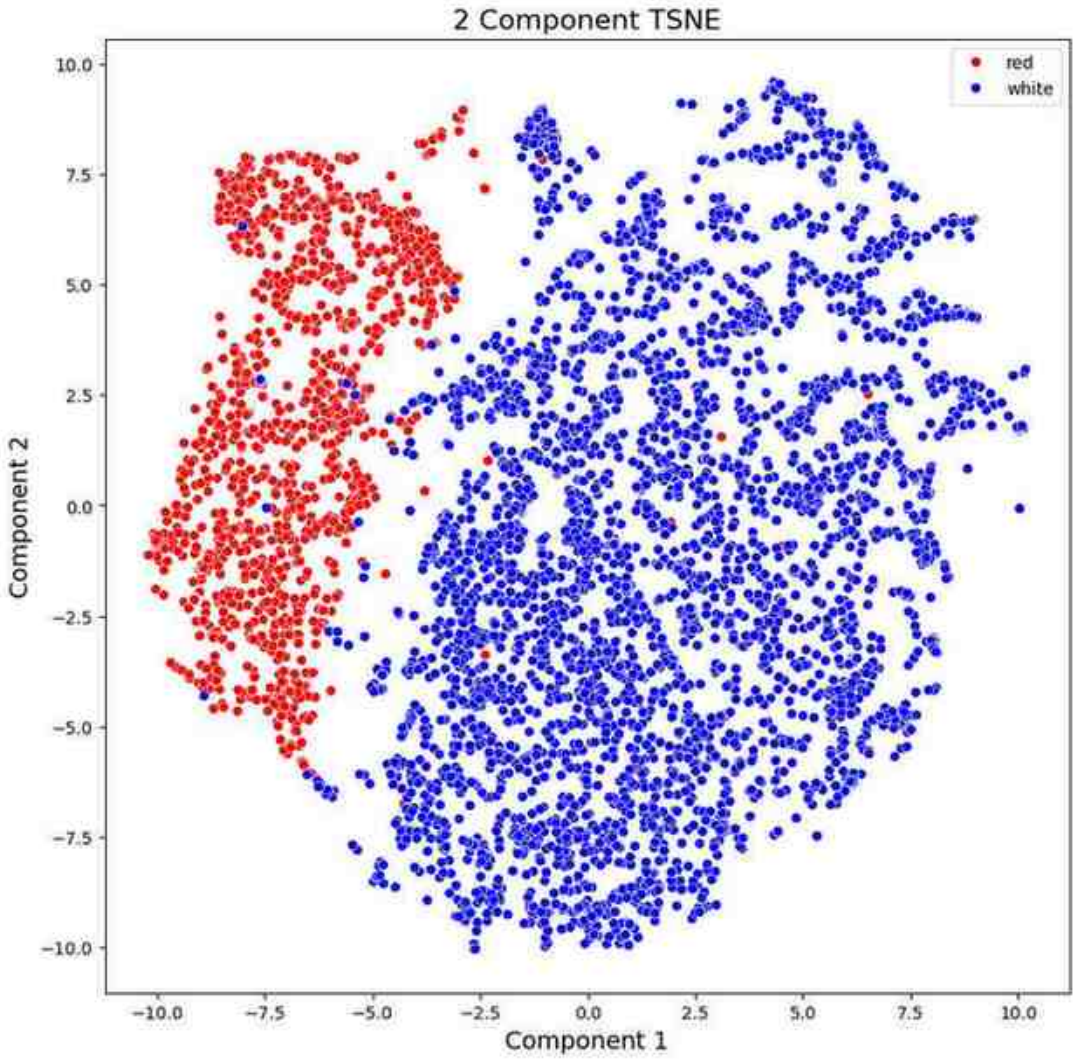


Figure 14: 2-dimensional t-SNE plot

3.9.3 UMAP (Uniform Manifold Approximation and Projection)

UMAP is a new technique published in 2018 for dimension reduction. This is an alternative to t-SNE for visualization, that preserves more of the global structure with better run time performance. The data is modeled by a manifold with a fuzzy topological structure. The small representation (or embedding) is found by searching for a low dimensional projection of the data that has the closest possible equivalent fuzzy topological structure.

```
import umap
data_umap = df_wine.copy()
data_umap = data_umap.drop(labels = ['quality', 'hue'],axis = 1)
scaled_data_umap = StandardScaler().fit_transform(data_umap) #We
    convert each feature into z-scores (number of standard
    deviations from the mean) for comparability.
umap = umap.UMAP()
umap_results = umap.fit_transform(scaled_data_umap)
umap_results

Output:
```

```
array([[ 2.0228717 , -0.65424883],
       [ 1.4935756 , -0.4979152 ],
       [ 1.3704098 , -0.3243352 ],
       ...,
       [11.418766  ,  7.4003463 ],
       [ 8.92388   , 12.713315  ],
       [ 8.568939  , 11.276885  ]], dtype=float32)
```

```
umap_dataset = pd.DataFrame(data = umap_results, columns = ['
    component1', 'component2'] )
umap_dataset['hue']=df_wine['hue']
plt.figure()
plt.figure(figsize=(10,10))
plt.xlabel('Component 1')
plt.ylabel('Component 2')
plt.title('2 Component UMAP')
sns.scatterplot(x = umap_dataset['component1'], y = umap_dataset[
    'component2'], hue = umap_dataset['hue'],alpha=1,palette=["red
    ", "blue"])
```

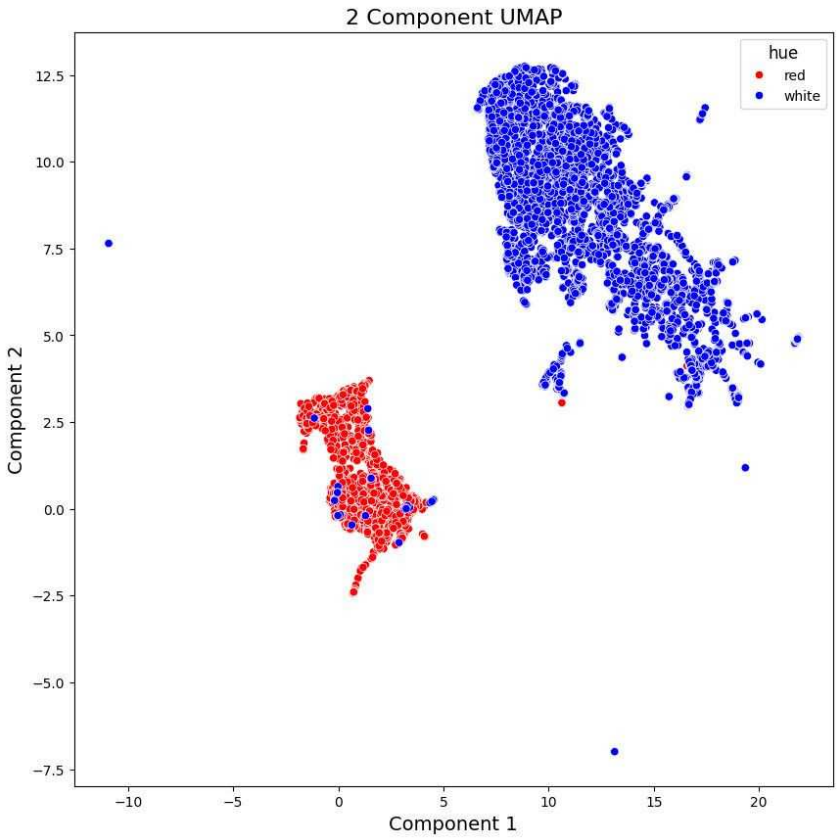


Figure 15: 2-dimensional UMAP plot

3.10 Classification

3.10.1 Logistic regression

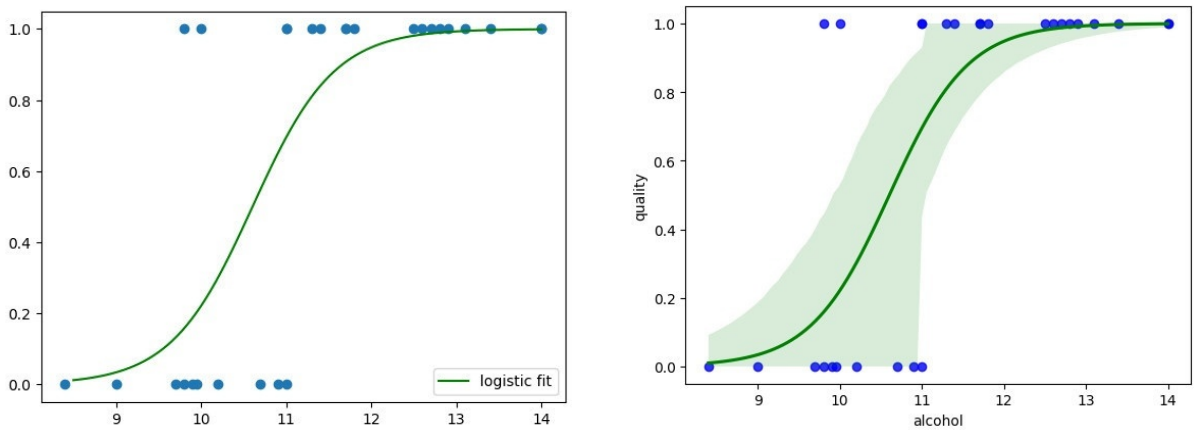
Logistic regression is a basic classification method. It is based on the logistic function which is a general form of the sigmoid function. It is an S-shaped curve that can take any real-valued number in the x-axis and values between 0 and 1 on the y axis. It can be used to classify between two classes (0 and 1). The y value expresses the probability that class 1 occurs.

```
df_LR = red_df.loc[(red_df['quality'] == 3 ) | (red_df['quality']
    == 8 ), ['alcohol','quality']]

df_LR.loc[df_LR.quality == 3, 'quality'] = 0
df_LR.loc[df_LR.quality == 8, 'quality'] = 1

df_LR.plot.scatter(x = 'alcohol', y = 'quality')
```

```
sns.regplot(x="alcohol", y="quality",data=df_LR, logistic=True,
   scatter_kws={'color': 'blue'}, line_kws={'color': 'green'})
plt.show()
```

```
from sklearn.linear_model import LogisticRegression
X = df_LR[['alcohol']]
y = df_LR.quality
model = LogisticRegression(C = 1e9)
model.fit(X, y)
```

```
from scipy.special import expit
x_test = np.linspace(8.5,14.0,100)
# predict dummy y_test data based on the logistic model
y_test = x_test * model.coef_ + model.intercept_
sigmoid = expit(y_test)
plt.scatter(X,y)
# ravel to convert the 2-d array to a flat array
plt.plot(x_test,sigmoid.ravel(),c="green", label = "logistic fit"
   )
plt.legend(loc="lower right")
```



(a) Logistic Regression plot (b) Logistic Regression plot

Figure 16: Hypothesis test plot

3.10.2 Data Splitting and Standardization

In general, the data is split into two groups before generating a model. These groups are called training and testing sets. The training set is used for developing models and select the parameters that are adequate for the data. The test set is used to assess the performance of the model with previously unseen data. The most common way is to divide the data is random splitting.

We will split the dataset using the function `train_test_split()` from scikit learn. The training set is 67% of the data and the test set is 33% (We have deliberately specified it that way in the parameters). The `random_state` parameter controls the shuffling applied to the data before applying the split.

Standardization is a method to transform variables that differ in mean and deviation into comparable values. This process consists of subtracting the means from each feature and then dividing by the feature standard deviation. Many machine learning algorithms assume that all features are centered around zero and have approximately the same variance, then standardization is needed.

In this case, we will use the function `StandardScaler()`. It will first fit the data to determine the mean and standard deviation and then transform it into a standardized form.

```
#Data splitting
from sklearn.model_selection import train_test_split
X = df_wine.drop(['quality', 'hue'], axis=1)
y = df_wine['hue']
X_train, X_test, y_train, y_test = train_test_split(X, y,
   test_size=0.33, random_state=42)
```

```
# Data Standardization
from sklearn import preprocessing
scaler = preprocessing.StandardScaler()
scaler.fit(X_train)
X_train = scaler.transform(X_train)
```

```
X_test = scaler.transform(X_test)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score
    , confusion_matrix

lr = LogisticRegression(random_state=0) #random_state parameter
    is provided to control the random number generator used
    whenever randomization is part of a Scikit-learn algorithm.
lr.fit(X_train, y_train)
```

3.10.3 Multiple Logistic Regression

In this case, we will use all the features with logistic regression to classify white and red wine.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, accuracy_score
    , confusion_matrix
lr = LogisticRegression(random_state=0) #random_state parameter
    is provided to control the random number generator used
    whenever randomization is part of a Scikit-learn algorithm.
lr.fit(X_train, y_train)
```

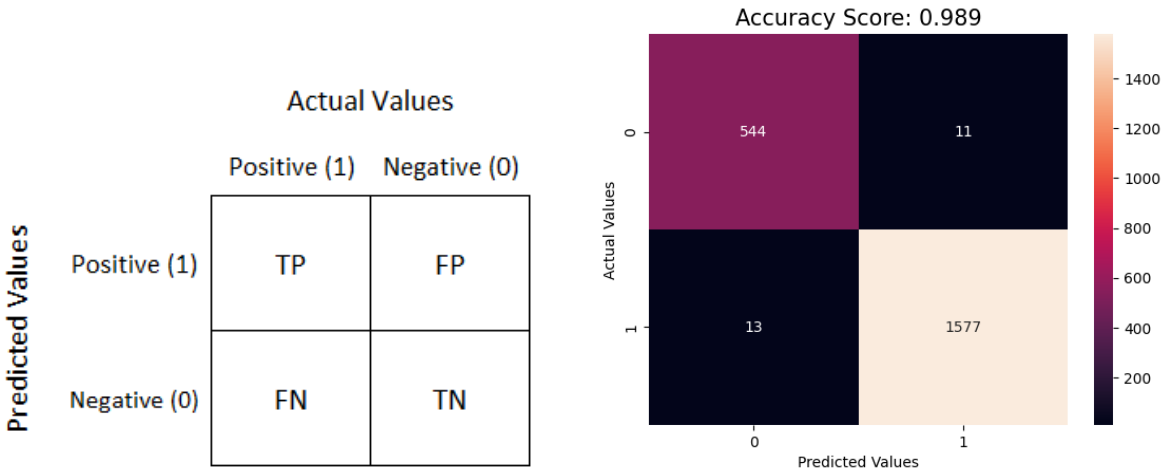
```
y_pred = lr.predict(X_test)
print("Accuracy score: " + str(accuracy_score(y_test, y_pred)))
print("\nConfusion matrix: \n" + str(confusion_matrix(y_test,
    y_pred)))
print("\nClassification report: \n" + str(classification_report(
    y_test, y_pred)))

Output:

Accuracy score: 0.9888111888111888

Confusion matrix:
[[ 544   13]
 [  11 1577]]

Classification report:
              precision    recall  f1-score   support

           0       0.98      0.98      0.98       557
           1       0.99      0.99      0.99      1588

    accuracy                           0.99      2145
   macro avg       0.99      0.98      0.99      2145
weighted avg       0.99      0.99      0.99      2145
```

3.11 Performance Metrics for Classification

The metrics are the values that guide the decision in machine learning. For example, choosing what model performs better or what parameters improve a model by measuring and comparing among different cases. In this section, we are going to analyze some of the most used metrics for the classification task.

Accuracy in this context of classification problems is the number of correct predictions over the total amount of predictions. This metric is useful when the classes to classify are approximately balanced. It is expected that the accuracy is better on the training set than on the test set.

```
train_accuracy = lr.score(X_train, y_train)
test_accuracy = lr.score(X_test, y_test)
print('One-vs-rest', '-'*35,
'Accuracy in Train Group   : {:.3f}'.format(train_accuracy),
'Accuracy in Test  Group   : {:.3f}'.format(test_accuracy), sep='
   \n')

Output:

One-vs-rest
-----------------------------------
Accuracy in Train Group   : 0.995
Accuracy in Test  Group   : 0.989
```



(a) Confusion matrix structure

(b) Confusion matrix of training dataset

3.11.1 Confusion Matrix

The Confusion Matrix is used for visualized the performance in a classification problem of two or more types of classes. For a binary case as ours, we compared the "Actual" vs "Predicted" values and obtain 4 cases: **TP** - True Positive **TN** - True Negative **FP** - False Positive **FN** - False Negative

```
from sklearn.metrics import confusion_matrix as cm
predictions = lr.predict(X_test)
```

```
score = round(accuracy_score(y_test, predictions), 3)
cm1 = cm(predictions, y_test)
sns.heatmap(cm1, annot=True, fmt=".0f")
plt.xlabel('Predicted Values')
plt.ylabel('Actual Values')
plt.title('Accuracy Score: {0}'.format(score), size = 15)
plt.show()
```

3.11.2 Precision score

The precision is a metric that informs how many positive-predicted instances were actually positive.

$$Precision = \frac{TP}{TP + FP}$$

```
from sklearn.metrics import precision_score
print("precision score: ", precision_score(y_test, predictions,
   average='micro'))

Output:
precision score:  0.9888111888111888
```

3.11.3 Recall score

The recall is a metric that informs how many instances were identified correctly of all the positive classes.

$$Recall = \frac{TP}{TP + FN}$$

```
from sklearn.metrics import recall_score
print("recall score: ", recall_score(y_test, predictions,
   average='micro'))

Output:
recall score:  0.9888111888111888
```

3.11.4 F1 score

The $F1 - Score$ is a metric that shows the harmonic mean of precision and recall.

$$F_1\ Score = \frac{2 \times Recall \times Precision}{Recall + Precision}$$

```
from sklearn.metrics import f1_score
precision_s = precision_score(y_test, predictions,average='micro'
   )
recall_s    = recall_score(y_test, predictions, average='micro')
print("F1_score     : ", 2*((precision_s*recall_s)/(precision_s
   + recall_s)))

Output:
F1_score:  0.9888111888111888
```

3.12 Decision tree

A decision tree is a non-linear technique that splits the dataset into subsets based on a feature value. This step is repeated recursively on each new subset until some criteria are reached. For example, every point in the subset has the same value of the target variable, or a maximum depth is attained. Choosing the feature and value to split is based on reducing the target entropy by dividing into pure subsets. An interesting feature of the decision trees is that we can generate an scheme that shows us how the decisions that were made.

```
from sklearn import tree
from sklearn.tree import DecisionTreeClassifier
dt = DecisionTreeClassifier(criterion = 'entropy', max_depth = 3,
    random_state = 1)
dt.fit(X_train, y_train)
y_pred = dt.predict(X_test)
print("Accuracy score: " + str(accuracy_score(y_test, y_pred)))
print("\nConfusion matrix: \n" + str(confusion_matrix(y_test,
   y_pred)))
print("\nClassification report: \n" + str(classification_report(
   y_test, y_pred)))

Output:
Accuracy score: 0.9724941724941725

Confusion matrix:
[[ 522   35]
 [  24 1564]]

Classification report:
              precision    recall  f1-score   support

           0       0.96      0.94      0.95       557
           1       0.98      0.98      0.98      1588

    accuracy                           0.97      2145
   macro avg       0.97      0.96      0.96      2145
weighted avg       0.97      0.97      0.97      2145
```

```
fig = plt.subplots(figsize=(20, 8))
tree.plot_tree(dt, fontsize=12)
```

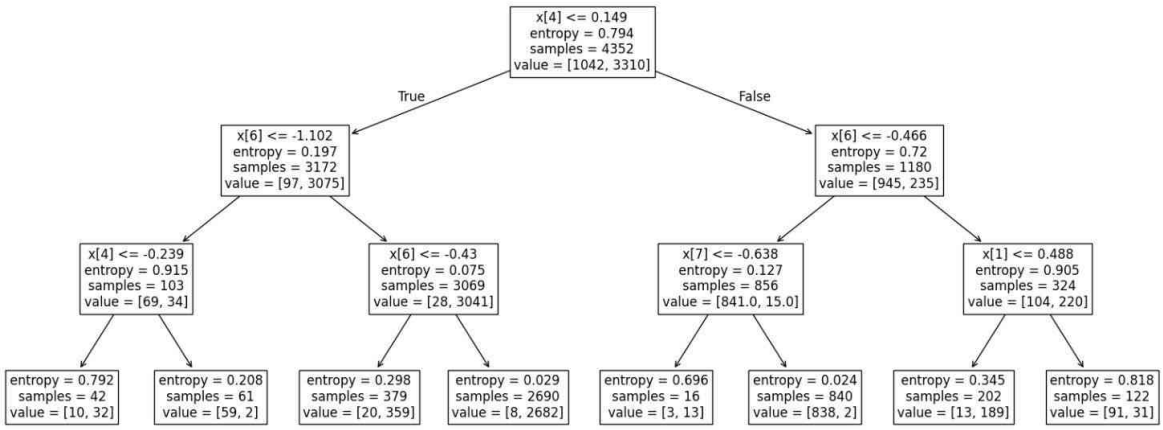


Figure 17: Decision tree plot for three generations.

3.13 Regression

3.13.1 Correlation

To fit the data with a linear model regression, it is a good practice to employ variables presenting a high correlation with the target. One way to do this is by calculating correlation coefficients and another way is through visual methods. We will predict the density for red wines, so we import the data and then inspect the dataset. From white wine correlation heatmap in fig. 10. We can look the variables which are related to density of wine.

3.13.2 Linear Regression

After the correlation analysis we will perform a linear regression using alcohol as the predictor variable (x_i) and density as the target (y_i), according to the model:

$$(y_i) = \beta_1 x_i + \beta_0$$

We will split the data into training and test sets, which is commonly done in machine learning methods for validation of the model created. It is the same method we used in section 3.10.2

```
from sklearn.linear_model import LinearRegression
linear_regression = LinearRegression()
linear_regression.fit(X = X_train, y = y_train)
# Let's print the parameters of the linear regression.
print('Beta1 = ' + str(linear_regression.coef_) + ', Beta0 = ' +
    str(linear_regression.intercept_))

Output:
Beta1 = [-0.00190561], Beta0 = 1.01408716780891
```

We can quantify how good the adjustment was by using the R^2 parameter. The adjustments usually are better on the training sets than the test sets, but in this case,

we will find that the training set has some outliers that make this adjustment for the training set worse.

$$R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}$$

```
from sklearn.metrics import r2_score
y_pred_test = linear_regression.predict(X_test)
y_pred_train = linear_regression.predict(X_train)
print('R2 train = ', r2_score(y_train, y_pred_train))
print('R2 test = ', r2_score(y_test, y_pred_test))

Output:
R2 train =  0.59265087229245
R2 test =  0.644777803115205
```

Visualizing result with scatterplot of predicted value and actual value.

```
plt.scatter(y_train,y_pred_train, label='Training Set')
plt.scatter(y_test,y_pred_test, label='Test Set')
plt.xlabel('Real')
plt.ylabel('Predicted')
plt.legend()
```

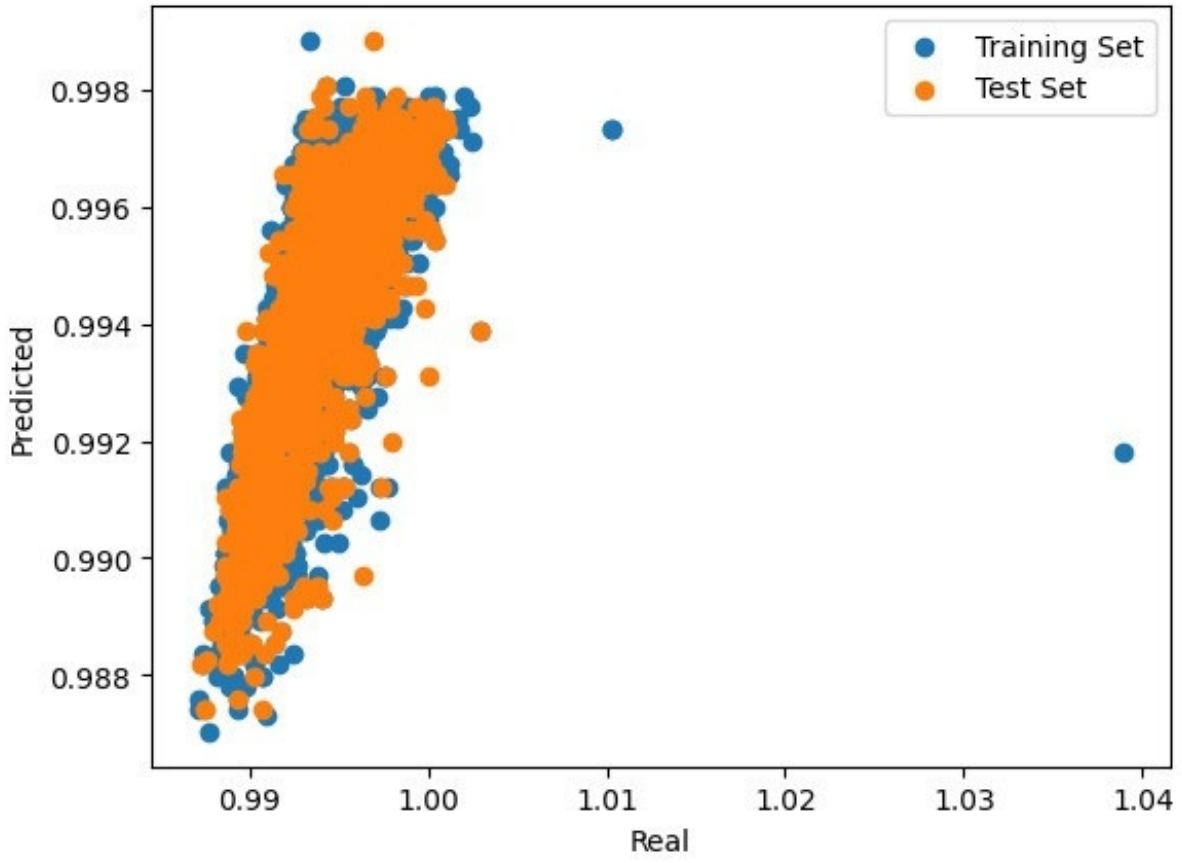


Figure 18: Linear regression plot between actual and predicted-trained data

3.13.3 Multiple Linear Regression

Multiple Linear Regression (MLR) is a generalization of classical linear regression. MLR models a linear relationship between the target response and multiple explanatory variables.

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_p x_{ip}$$

Since MLR is a generalization, the Scikit Learn library uses the same function that we used before for Linear Regression. So training and linear regression uses the same method from section 3.13.2

```
from sklearn.linear_model import LinearRegression
multiple_linear_regression = LinearRegression()
multiple_linear_regression.fit(X = X_train, y = y_train)
```

Increasing the number of predictor variables leads to a better adjustment of the target so that the value of R^2 increases.

```
from sklearn.metrics import r2_score
pred_train_lr = multiple_linear_regression.predict(X_train)
pred_test_lr = multiple_linear_regression.predict(X_test)
print('R2 training = ', r2_score(y_train, pred_train_lr))
print('R2 test = ', r2_score(y_test, pred_test_lr))

Output:
R2 training =  0.9608334119853044
R2 test =  0.9724221101555639
```

3.13.4 Root Mean Squared Error (RMSE)

Another useful metric is Root Mean Squared Error (RMSE). It has the advantage that it can be used for non-linear models.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

```
from sklearn.metrics import mean_squared_error
rmse_test = np.sqrt(mean_squared_error(y_test,pred_test_lr))
print('RSME test= ', rmse_test)

Output:
RSME test=  0.00047624213322229874
```

3.14 LASSO(Least Absolute Shrinkage and Selection Operator)

Least Absolute Shrinkage and Selection Operator (LASSO) is a linear regression method that performs variable selection and regularization to improve prediction accuracy and

produce a simpler model. This method uses a cost function with a constant α that determines the degree of penalization. Scikit-learn has implemented LASSO in the function `sklearn.linear_model.Lasso`.

$$\text{LASSO}_{\text{Cost Function}} = \sum_{i=1}^{M}(y_i - \hat{y}_i)^2 = \sum_{i=1}^{M}\left(y_i - \sum_{j=0}^{p} w_j x_{ij}\right)^2 + \alpha \sum_{j=0}^{p} |w_j|$$

$$\text{For some } t > 0, \quad \sum_{j=0}^{p} |w_j| < t$$

```
from sklearn.linear_model import Lasso
lasso_regression = Lasso(alpha=0.0001)
lasso_regression.fit(X = X_train, y = y_train)
```

```
from sklearn.metrics import r2_score, mean_squared_error
y_pred = lasso_regression.predict(X_test)
r2 = r2_score(y_test, y_pred)
print('R2 test = ', r2)

Output:
R2 test =  0.9680058702120279
```

Let's analyze if using LASSO we have a smaller model where some of the coefficients are zero.

```
coeffecients = pd.DataFrame(lasso_regression.coef_,X.columns.
   tolist())
coeffecients.columns = ['Coeffecient']
print(coeffecients)

Output:
                      Coeffecient
fixed acidity            0.000518
volatile acidity         0.000000
citric acid              0.000000
residual sugar           0.001855
chlorides                0.000027
free sulfur dioxide     -0.000000
total sulfur dioxide     0.000090
pH                       0.000345
sulphates                0.000096
alcohol                 -0.001337
```

```
coefficients.plot.bar()
```

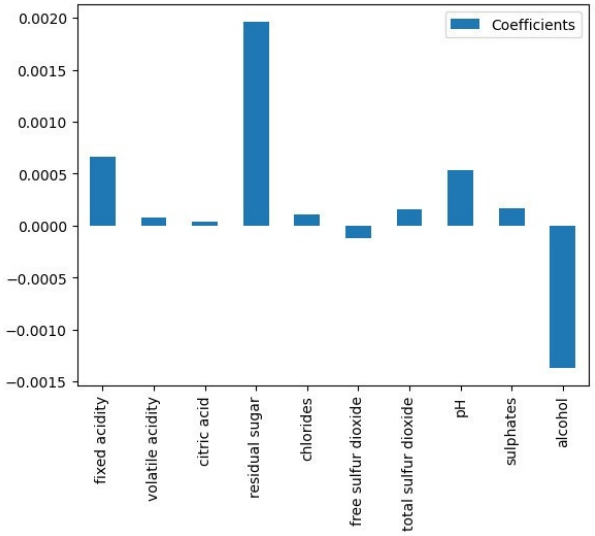


Figure 19: Bar plot of coefficients analyzed using LASSO

Finally, we will analyze how the RSME of the training and test sets change for different alpha values.

```
alphas=np.logspace(-10,3,endpoint=True,num=100,base=10)
RMSE=[]
RMSE_p=[]
for x in (alphas):
    #print(x)
    model_lasso = Lasso(x)
    model_lasso.fit(X_train, y_train)
    pred_test_lasso= model_lasso.predict(X_test)
    pred_train_lasso=model_lasso.predict(X_train)
    RMSE_p.append(np.sqrt(mean_squared_error(y_test,
       pred_test_lasso)))
    RMSE.append(np.sqrt(mean_squared_error(y_train,
       pred_train_lasso)))
fig=plt.gcf()
fig.set_size_inches(12,7)
plt.plot(alphas,RMSE, label='Training Set', linewidth=6)
plt.plot(alphas,RMSE_p, label='Test Set',linewidth=6)
plt.plot(alphas,len(alphas)*[rmse_MLR], label='MLR',linewidth=4)
plt.xscale("log")
plt.xlabel('alpha')
plt.ylabel('RMSE')
plt.legend()
```

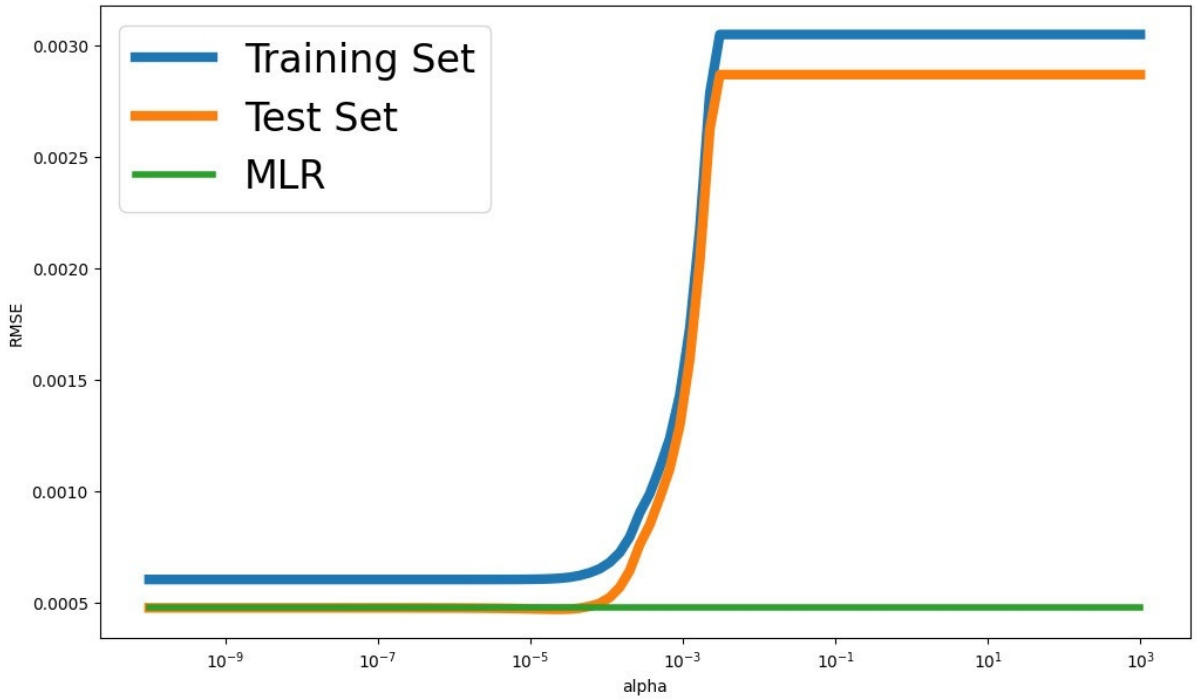


Figure 20: Bar plot of coefficients analyzed using LASSO

3.15 Discussion and Conclusion

From the analysis of red & white wine-quality dataset. We can conclude these difference between red and white wine - Red wines exhibit higher levels of volatile acidity, sulphates & chlorides content, and density. In contrast, white wines show significantly higher residual sugar and sulfur dioxide content which indicates a sweeter and more preserved character. Additionally, white wines tend to have slightly higher average quality ratings, possibly due to their cleaner fermentation and lower acidity.

This study demonstrates that machine learning is a powerful tool for chemists, enabling efficient analysis, visualization, and interpretation of experimental data. Through hands-on work with five Python notebooks, covering topics from classification and regression to data visualization, participants applied ML techniques to a real-world wine dataset.

The project shows that even small datasets can benefit from data-driven approaches, as illustrated in section 2 Integrating machine learning into chemical research enhances data handling and supports better scientific conclusions. Overall, ML is becoming an essential skill for chemists, streamlining research and accelerating discovery.

Appendix

1. Supplementary material of project 1 containing theatrical data used for calculating percentage error.
2. Red and White wine-quality dataset.
3. 1^{st} experiment-Aromaticity dataset Google-sheets
4. 2^{nd} experiment-Acid Base Dataset Google-Sheets
5. Computational chemistry github. Molecule construction, input & output files of 1^{st} and 2^{nd}.
6. Machine Learning for chemist. 3^{rd} experiment's worked-out Assignments and Jupyter Notebooks.
7. Google-colab Interactive Python Notebooks
8. ***mhchem*** Chemistry Equation LaTeXpackage. Chemical equations in the report used this package.
9. ***chemfig*** Chemistry structures drawing LaTeXpackage. Cyclic Organic Chemical figures in this report are build using this package.

References

[1] Wayne P. Anderson. A lewis acid-base computational chemistry exercise for advanced inorganic chemistry. *Journal of Chemical Education*, 77(2):209–214, 2000.

[2] J. Klapper, D. CP Magill, and T. JW Greenbowe. Quantitative thermodynamic descriptions of aromaticity: A computer-driven class experiment. *Journal of Chemical Education*, 82(6):953–958, 2005.

[3] Deborah Lafuente, Brenda H. Cohen, Guillermo Fiorini, Agustín A. García, Mauro Bringas, Ezequiel Morzan, and Diego Onna. A gentle introduction to machine learning for chemists: An undergraduate workshop using python notebooks for visualization, data processing, analysis, and modeling. *Journal of Chemical Education*, 98(9):2892–2898, 2021.